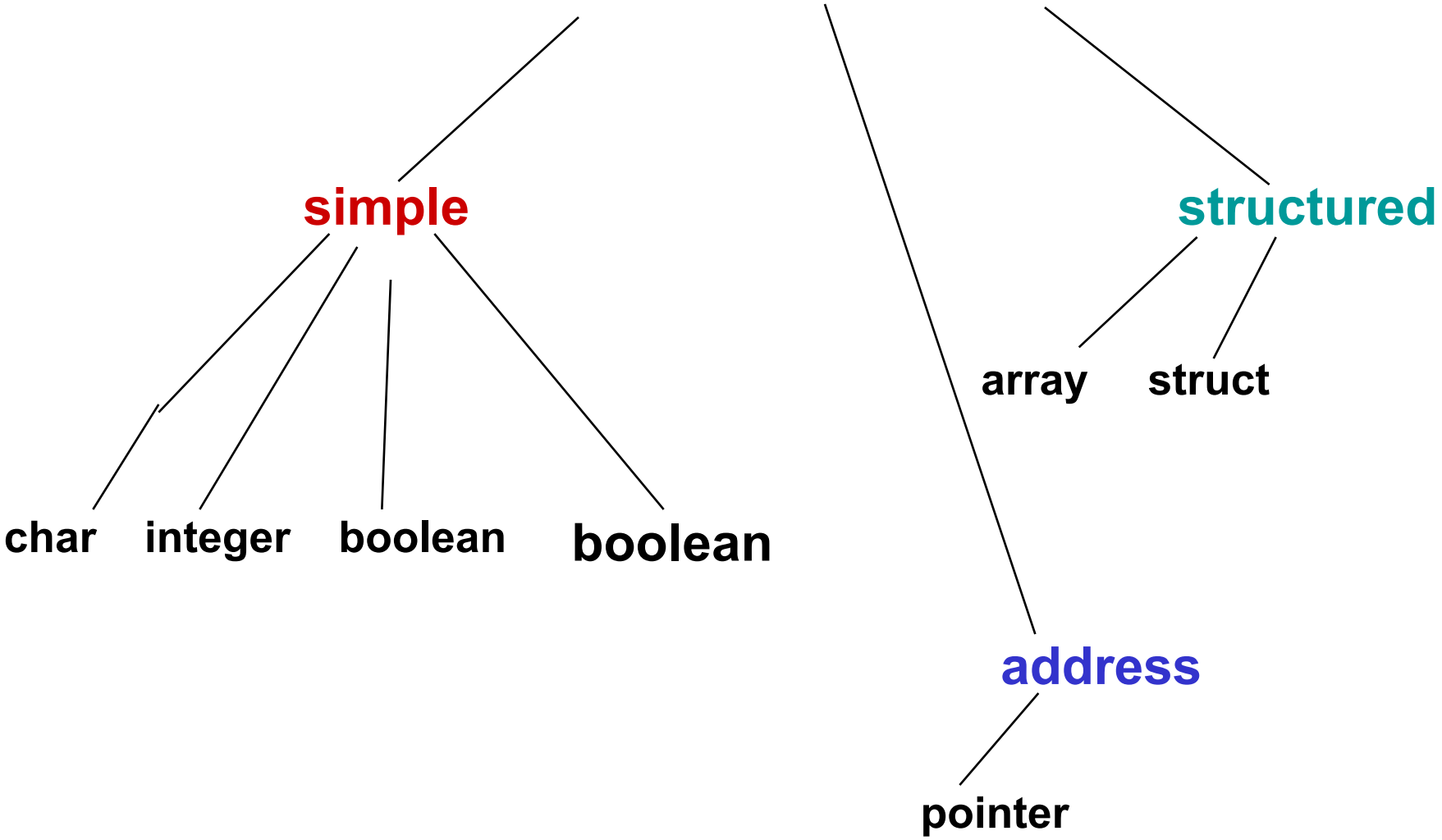
Pointers, Dynamic Data, and Reference Types

- Review on Pointers
- Reference Variables
- Dynamic Memory Allocation
  - The new
  - The dispose
  - Dynamic Memory Allocation for Arrays

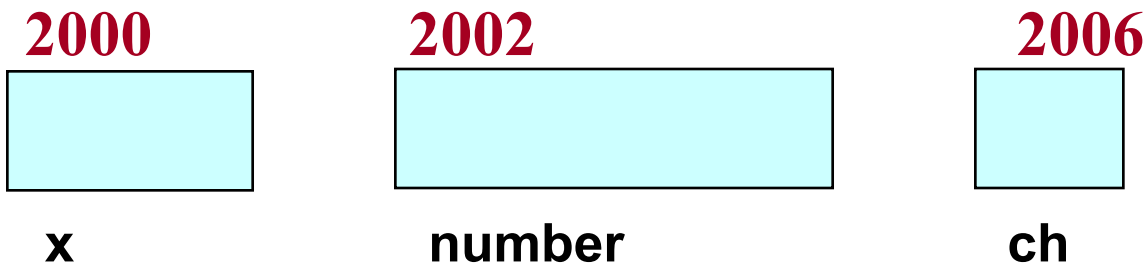
Pascal Data Types



Addresses in Memory

- When a variable is declared, enough memory to hold a value of that type is allocated for it at an unused memory location. This is the address of the variable

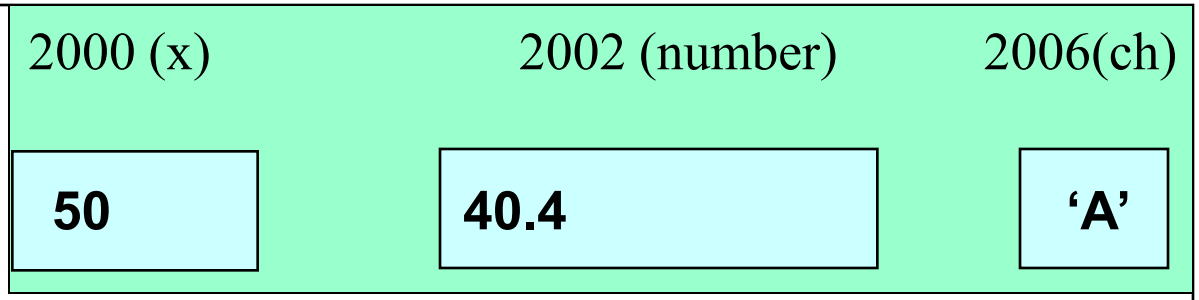
```
var    x:integer;
       number:real;
       ch:char;
```



Pascal : "@" address operator

- pascal supports "operator" that is able to gets a memory address of variable , the address-of operator @

```
X:integer;
Number:real;
Ch:char;
```




```
Writeln('x= ', x, ' , addr x= `, HexStr(@x));
Writeln('number= ', number, ' , addr number= HexStr(@number));
Writeln('ch= ', ch, ' , addr ch= ', HexStr(@ch));
```

printAdr.pas

```
program PrintVariablesAndAddresses;

var
  x: Integer;
  number: real;
  ch: Char;

begin
  x := 50;
  number := 40.4;
  ch := 'A';

  Writeln('x= ', x, ' , addr x= ', HexStr(@x));
  Writeln('number= ', number:0:6, ' , addr number= ', HexStr(@number));
  Writeln('ch= ', ch, ' , addr ch= ', HexStr(@ch));
end.

```

```
x= 50 , addr x= 000000000042D480
number= 40.400000 , addr number= 000000000042D490
ch= A , addr ch= 000000000042D4A0
```

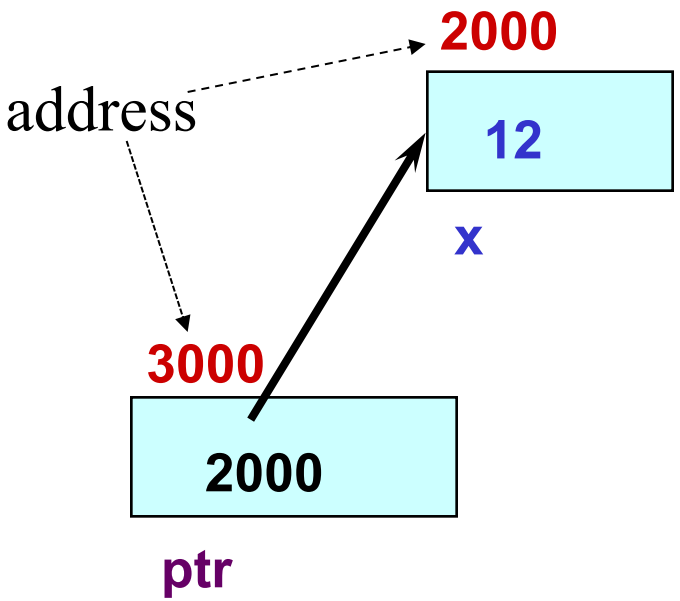
What is a pointer variable?

- Pointer data type is a special kind of variable which are meant to store addresses only, instead of values(integer, float, double, char, etc).
- A pointer variable is a variable whose value is the address of a location in memory.
- To declare a pointer variable, you must specify the type of value that the pointer will point to, for example
- ใช้สัญลักษณ์ ^

```
var
  ptr: ^Integer; // ptr will hold the address of an Integer
  q: ^Char;      // q will hold the address of a Char
  e: ^employee;   // e will hold the address of an employee
                  //record
```

Using a Pointer Variable

```
x:integer;
ptr:^Integer;
x := 12;
ptr:= @x;
wrieln(ptr)-> ?
wrieln(@ptr)-> ?
```



NOTE: **Because ptr holds the address of x,**

we say that ptr "points to" x

```
1  program PrintVariablesAndAddresses;
2
3  var
4    x: Integer;
5    ptr: ^Integer;
6  begin
7    x := 50;
8    ptr := @x;
9
10   Writeln('x= ', x, ' , addr x= ', PtrUInt(@x) );
11   Writeln('ptr = ', PtrUInt(ptr) ,
12                ' , addr ptr= ', PtrUInt(@ptr) );
13 end.
14
```

prnAddr.pas

Output:

```
x= 50 , addr x= 4379728
ptr = 4379728 , addr ptr= 4379744
```

ขนาด memory ของ pointer

szptr.pas

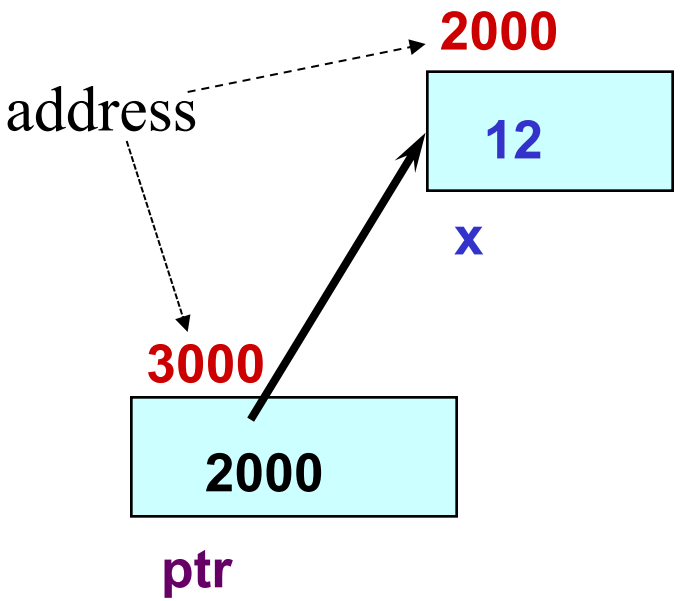
```
1  program SizeOfExample;
2
3  uses
4    SysUtils;
5
6  type
7    employee = record
8      name: array[0..15] of Char;
9      id: Integer;
10   end;
11
12 var
13   i: Integer;
14   pi: ^Integer;
15
16   f: real;
17   pf: ^real;
18
19   c: Char;
20   pc: ^Char;
21
22   d: Double;
23   pd: ^Double;
24
25   e: employee;
26   pe: ^employee;
27
28 begin
29   Writeln('sizeof(i) = ', SizeOf(i), ' sizeof(pi) = ', SizeOf(pi));
30   Writeln('sizeof(f) = ', SizeOf(f), ' sizeof(pf) = ', SizeOf(pf));
31   Writeln('sizeof(c) = ', SizeOf(c), ' sizeof(pc) = ', SizeOf(pc));
32   Writeln('sizeof(d) = ', SizeOf(d), ' sizeof(pd) = ', SizeOf(pd));
33   Writeln('sizeof(e) = ', SizeOf(e), ' sizeof(pe) = ', SizeOf(pe));
34 end.
35
```

Output:

```
sizeof(i) = 2 sizeof(pi) = 8
sizeof(f) = 8 sizeof(pf) = 8
sizeof(c) = 1 sizeof(pc) = 8
sizeof(d) = 8 sizeof(pd) = 8
sizeof(e) = 18 sizeof(pe) = 8
```

^: dereference operator (1.1)

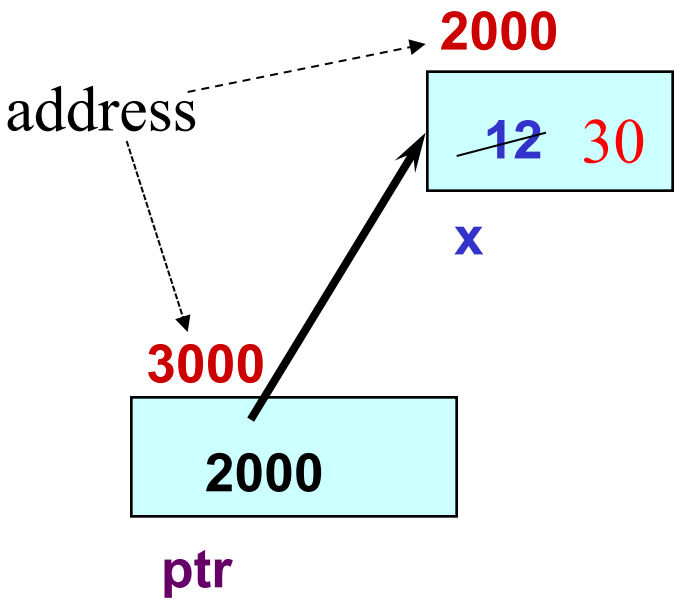
```
x:integer;
ptr:^Integer;
x := 12;
ptr:= @x;
write(ptr^) -> ?
ptr^:= 30;
write(ptr^)
```



NOTE: **Because ptr holds the address of x,**
we say that ptr "points to" x

^: dereference operator (1.2)

```
x:integer;
ptr:^Integer;
x := 12;
ptr:= @x;
write(ptr^)
ptr^:= 30;
write(ptr^) -> ?
```



NOTE: **Because ptr holds the address of x, we say that ptr "points to" x**

Ptrval.pas

Output:

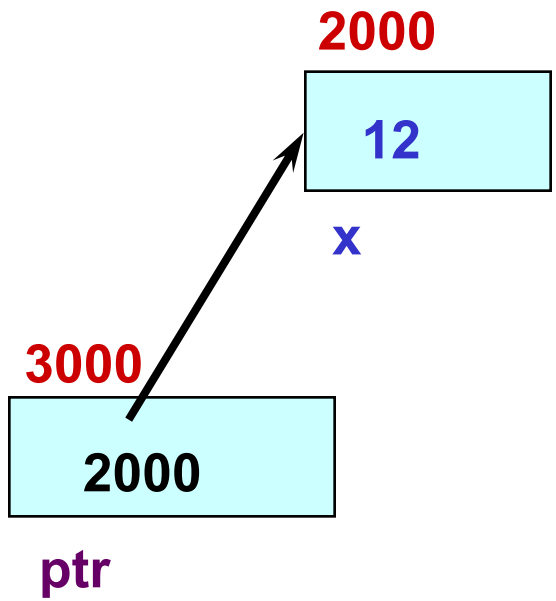
```
x= 50 , addr x= 00472430
ptr = 00472430 , addr ptr= 00472440
```

```
1  program ptrval;
2
3  var
4    x: Integer;
5    ptr: ^Integer;
6  begin
7    x := 50;
8    ptr := @x;
9
10   Writeln('x= ', x, ' , addr x= ', HexStr(@x));
11   Writeln('ptr = ', HexStr(ptr), ' , addr ptr= ', HexStr(@ptr));
12   Writeln('ptr^ = ',ptr^ );
13   ptr^ := 60;
14   Writeln('x= ', x, ' , ptr^= ', ptr^);
15 end.
16
```

```
x= 50 , addr x= 00472430
ptr = 00472430 , addr ptr= 00472440
ptr^ = 50
x= 60 , ptr^= 60
```

^: dereference operator (2.1)

```
x:integer;
ptr:^integer;
 x := 12;
 y := 10;
 ptr := @x;
 ptr := @y;
 ptr^:= 5;
```

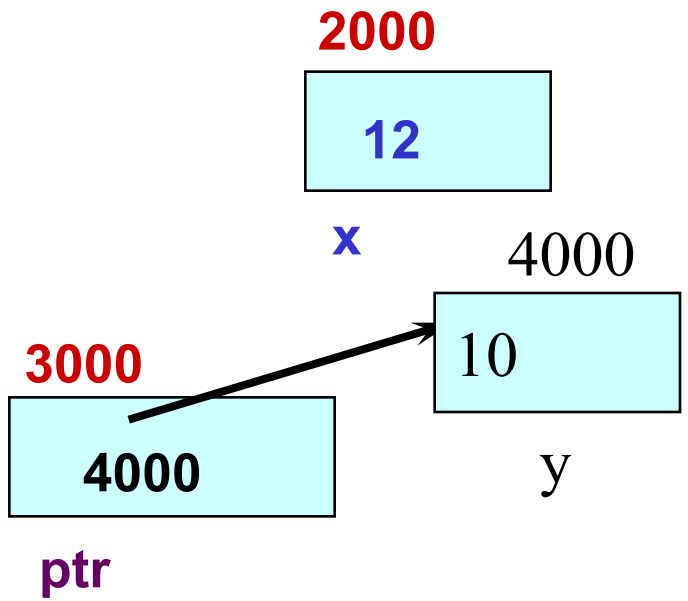


NOTE: **The value pointed to by ptr is denoted by ^ptr**

* (deference operator) การ access ข้อมูลที่ pointer ชี้อยู่

^: dereference operator (2.2)

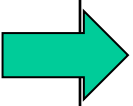
```
x:integer;
ptr:^integer;
 x := 12;
 y := 10;
 ptr  := @x;
 ptr  := @y;
 ptr^ := 5;
```

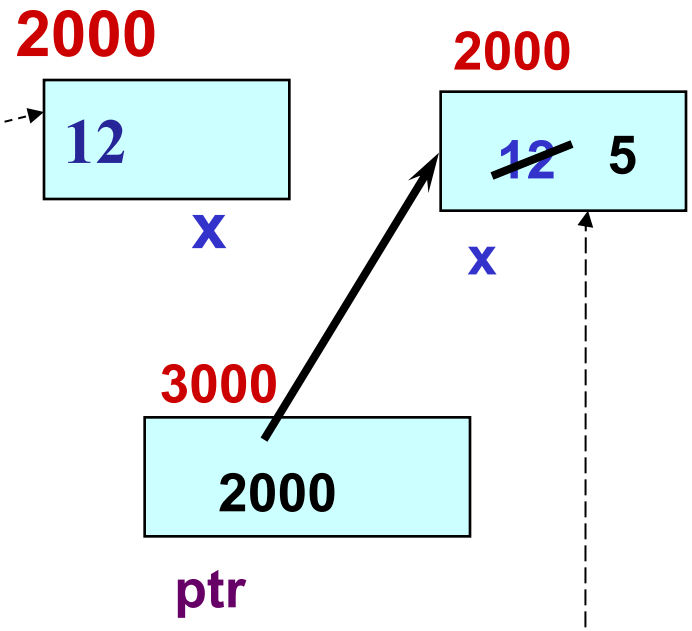


* (deference operator) การ access ข้อมูลที่ pointer ชี้อยู่

^: dereference operator (2.3)

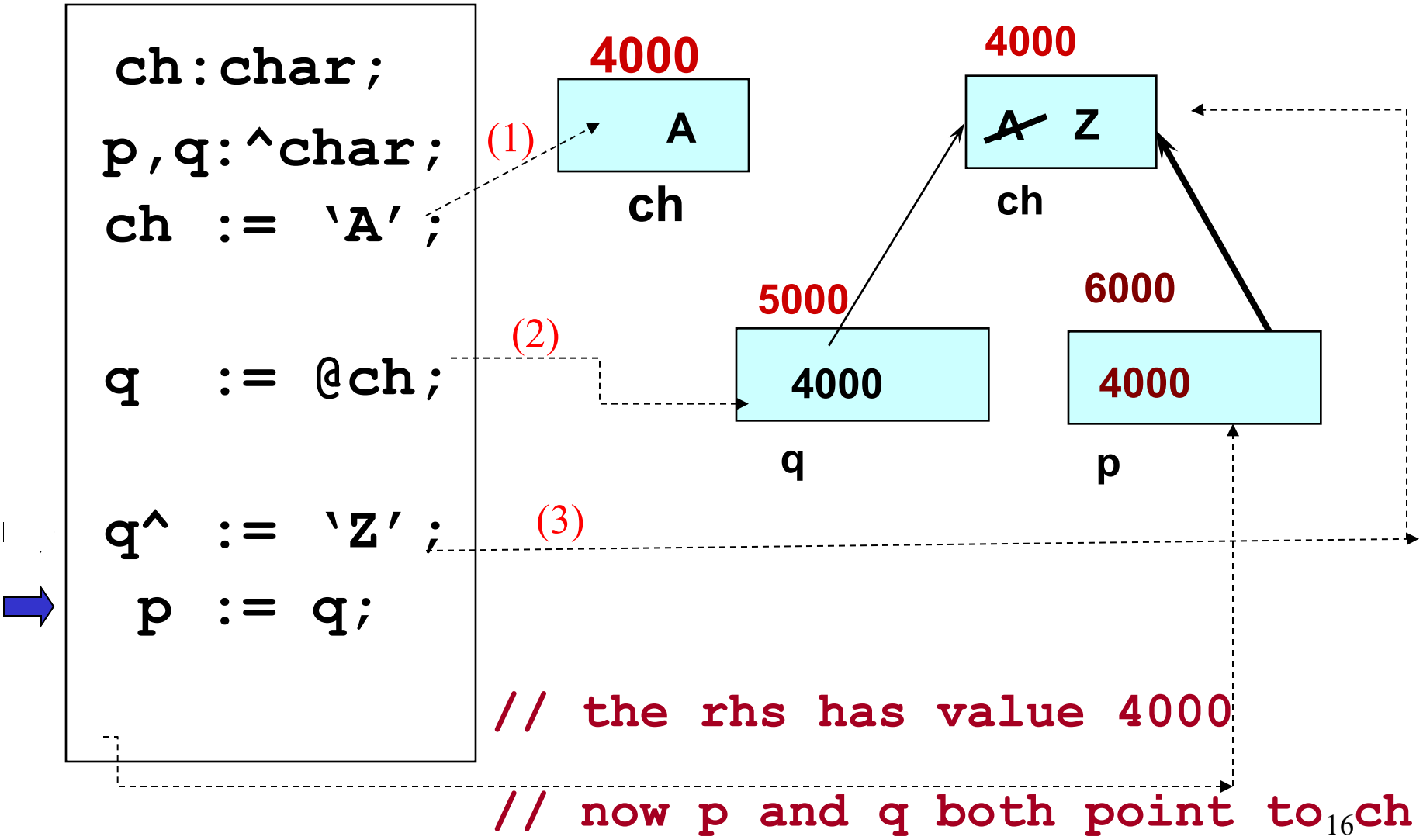
```
x:integer;
ptr:^integer;
 x := 12;
 y := 10;
 ptr  := @x;
 ptr  :=  @y;
 ptr^ :=  5;
```






```
// changes the value at the
   address ptr points to 5
```

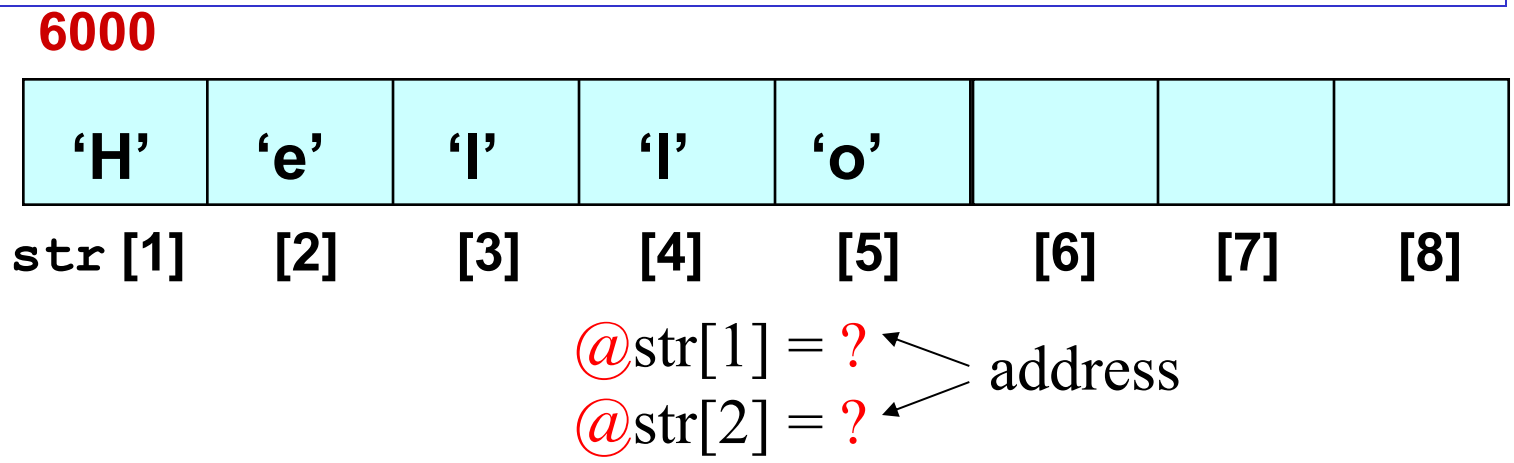
Self –Test on Pointers



Pointer Arithmetic and Arrays

str :string[8]; or str:array[1..8]

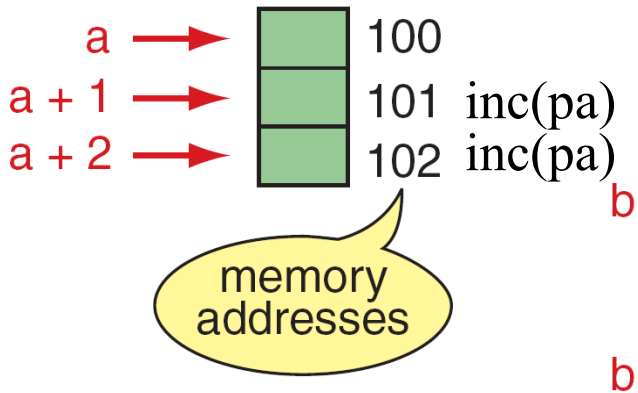
- **`str`** is the base address of the array.
- We say **`str`** is a pointer because its value is an address.
- It is a pointer constant because the value of **`str`** itself cannot be changed by assignment. It "points" to the memory location of a **`char`**.

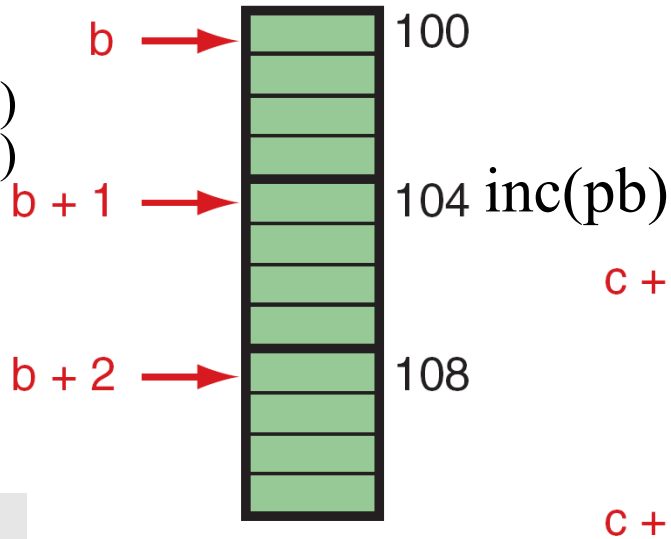


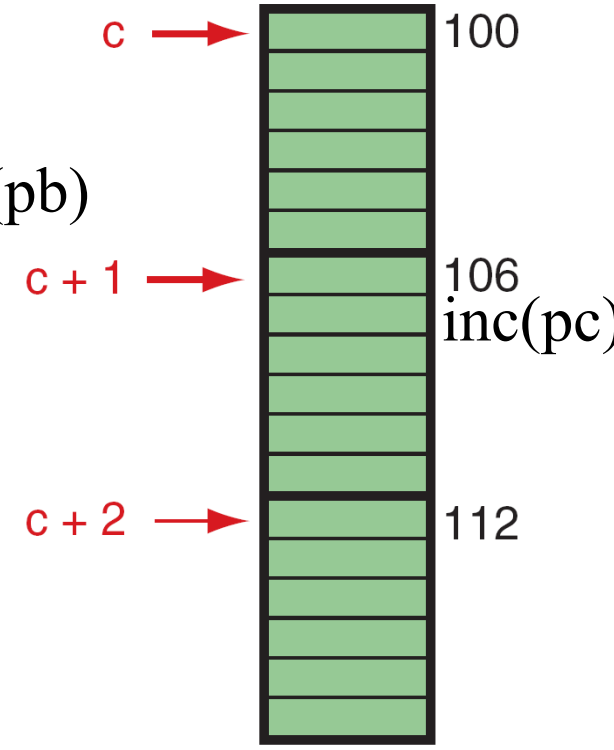
```
1  program PrintArryAdr;
2
3  var
4    s1:string[8];
5    s2:array [1..8] of char ;
6  begin
7
8    s1 := 'A';
9    s2[1] := 'B';
10   Writeln( 's1 = ',s1[1]);
11   Writeln( 's2 = ',s2[1]);
12
13   Writeln( '@s1[1] = ', PtrUInt( @s1[1] ) );
14   Writeln( '@s1[2] = ', PtrUInt( @s1[2] ) );
15
16   Writeln( '@s2[1] = ', PtrUInt( @s2[1] ) ) ;
17   Writeln( '@s2[2] = ', PtrUInt( @s2[2] ) ) ;
18
19   .
20 end.
```

```
21 lines compiled, 0.0 s
s1 = A
s2 = B
@s1[1] = 4384817
@s1[2] = 4384818
@s2[1] = 4384832
@s2[2] = 4384833
```

Given pointer, p, inc(p) & dec(p) is a pointer to the value n elements away.








```
a : string[3];
b:array[1..3]of integer;
c :array[1..3]of real;
```

```
pa  = @a[0]
pb  = @b[0]
pc  = @c[0]
```

FIGURE 10-6 Pointer Arithmetic and Different Types

HelloWorld.pas

```
1  program incDecPtr;
2
3  uses
4    SysUtils;
5
6  var
7    str: string;
8    p: ^Char;
9  begin
10   str := 'Hello';  // You must initialize the string before accessing characters
11
12   Writeln('sizeof(str) = ', SizeOf(str), ',str=',str);
13   Writeln('sizeof(str[1]) = ', SizeOf(str[1]), ' , @str[1] = ', PtrUInt(@str[1]));
14   Writeln('sizeof(str[2]) = ', SizeOf(str[2]), ' , @str[2] = ', PtrUInt(@str[2]));
15
16   p := @str[1];  // Use := for assignment, and point to first character of the string
17   Writeln('p = ', PtrUInt(p), ' , ^p = ', p^, ' , @p = ', PtrUInt(@p));
18   Inc(p);
19   Writeln('p = ', PtrUInt(p), ' , ^p = ', p^, ' , @p = ', PtrUInt(@p));
20   Dec(p);
21   Writeln('p = ', PtrUInt(p), ' , ^p = ', p^, ' , @p = ', PtrUInt(@p));
22
23 end.
24
```

Output:

```
sizeof(str) = 256,str=Hello
sizeof(str[1]) = 1 , @str[1] = 4662321
sizeof(str[2]) = 1 , @str[2] = 4662322
p = 4662321 , ^p = H , @p = 4662576
p = 4662322 , ^p = e , @p = 4662576
p = 4662321 , ^p = H , @p = 4662576
```

incDecPtr.pas

Inc(p)
Dec(p)

```
1  program incDecPtrInt;
2
3  uses
4    SysUtils;
5
6  var
7    arr: array[1..5] of Integer;
8    p: ^Integer;
9  begin
10   // Initialize the array
11   arr[1] := 10;
12   arr[2] := 20;
13   arr[3] := 30;
14   arr[4] := 40;
15   arr[5] := 50;
16
17   Writeln('sizeof(arr) = ', SizeOf(arr));
18   Writeln('sizeof(arr[1]) = ', SizeOf(arr[1]), ' , @arr[1] = ', PtrUInt(@arr[1]));
19   Writeln('sizeof(arr[2]) = ', SizeOf(arr[2]), ' , @arr[2] = ', PtrUInt(@arr[2]));
20
21   p := @arr[1];  // Pointer to first element
22   Writeln('1.p = ', PtrUInt(p), ' , ^p = ', p^, ' , @p = ', PtrUInt(@p));
23   Inc(p);  // Pointer to first element
24   Writeln('2.p = ', PtrUInt(p), ' , ^p = ', p^, ' , @p = ', PtrUInt(@p));
25   Dec(p);  // Pointer to first element
26   Writeln('3.p = ', PtrUInt(p), ' , ^p = ', p^, ' , @p = ', PtrUInt(@p));
27
28 end.
29
```

Output:

```
sizeof(arr) = 10
sizeof(arr[1]) = 2 , @arr[1] = 4662320
sizeof(arr[2]) = 2 , @arr[2] = 4662322
1.p = 4662320 , ^p = 10 , @p = 4662336
2.p = 4662322 , ^p = 20 , @p = 4662336
3.p = 4662320 , ^p = 10 , @p = 4662336
```

incDecPtrInt.pas

Inc(p)
Dec(p)

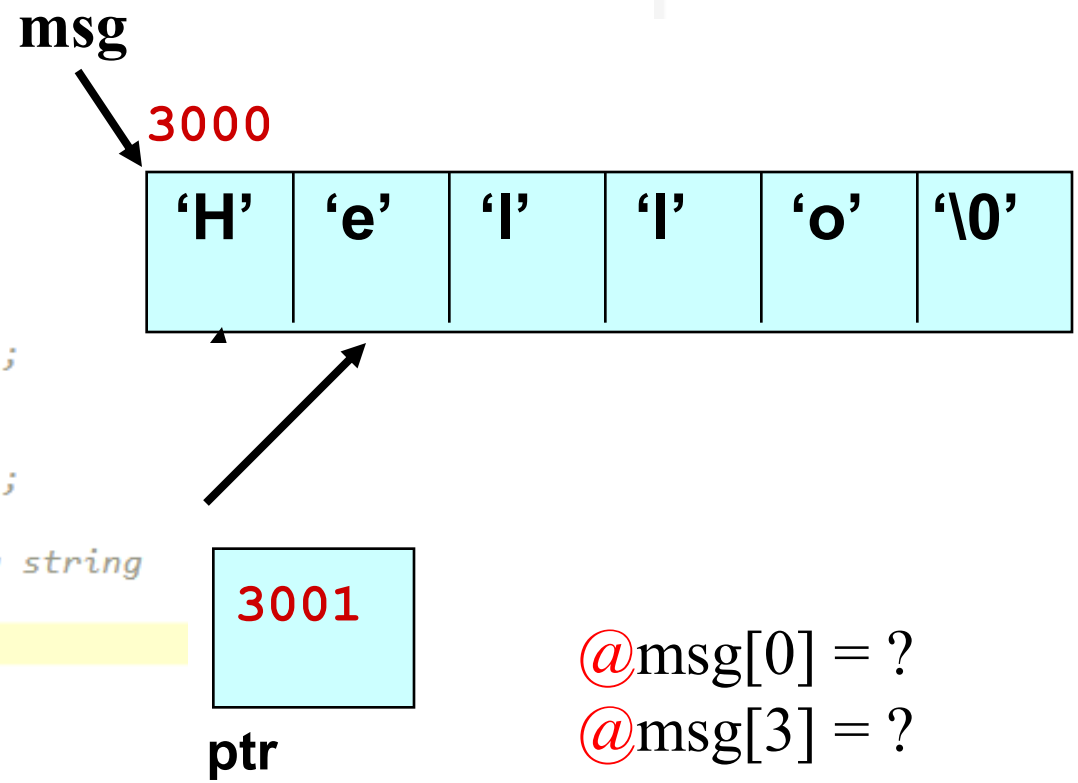
Pointer Arithmetic and Arrays (3.1)

```
1  program CharPointerExample;
2
3  uses
4    SysUtils;
5
6  var
7    msg: string;
8    ptr: ^Char;
9  begin
10   msg := 'Hello';  // 5 chars
11
12   Writeln(' msg: ', msg);
13
14   ptr := @msg[1];     // ptr = msg;
15
16   ptr^ := 'M';        // *ptr = 'M';
17   Inc(ptr);           // ptr++;
18
19   ptr^ := 'a';        // *ptr = 'a';
20
21   // Print the modified array as a string
22   Writeln('Modified msg: ', msg);
23 end.
24
```

chpoint.pas

Output:

```
 msg: Hello
Modified msg: Mallo
```



@msg[0] = ?

@msg[3] = ?

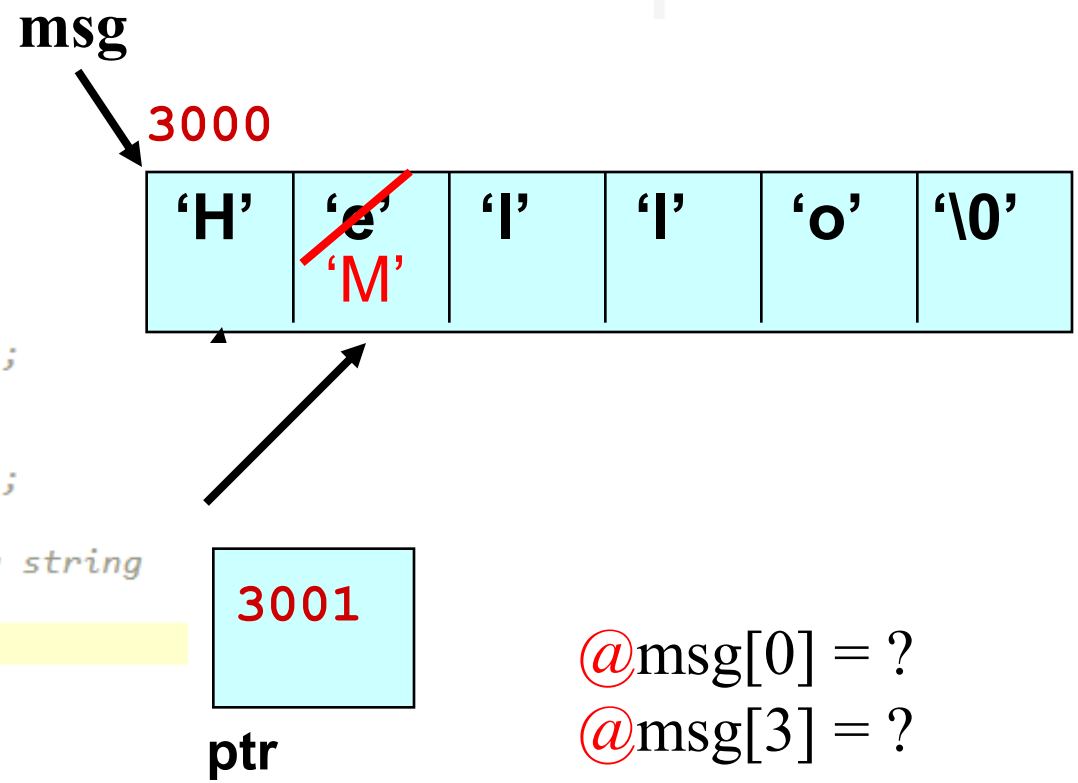
Pointer Arithmetic and Arrays (3.2)

```
1  program CharPointerExample;
2
3  uses
4    SysUtils;
5
6  var
7    msg: string;
8    ptr: ^Char;
9  begin
10   msg := 'Hello';  // 5 chars
11
12   Writeln(' msg: ', msg);
13
14   ptr := @msg[1];     // ptr = msg;
15
16   ptr^ := 'M';        // *ptr = 'M';
17   Inc(ptr);           // ptr++;
18
19   ptr^ := 'a';        // *ptr = 'a';
20
21   // Print the modified array as a string
22   Writeln('Modified msg: ', msg);
23 end.
24
```

chpoint.pas

Output:

```
 msg: Hello
Modified msg: Mallo
```



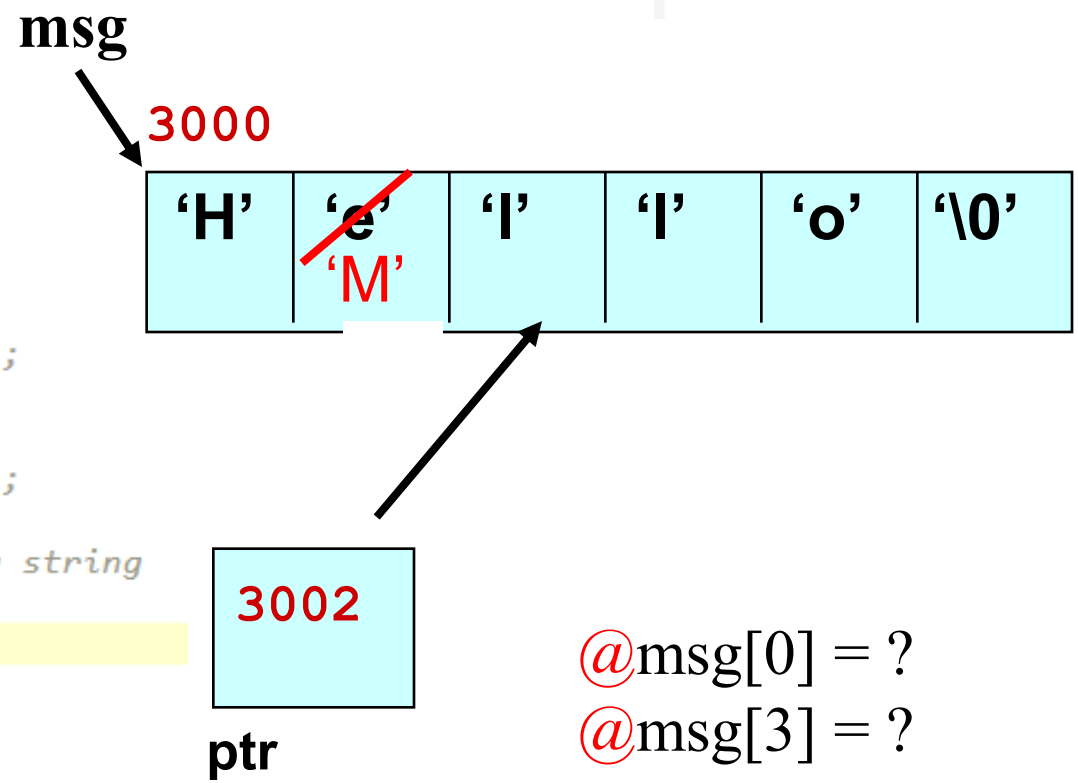
Pointer Arithmetic and Arrays (3.3)

```
1  program CharPointerExample;
2
3  uses
4    SysUtils;
5
6  var
7    msg: string;
8    ptr: ^Char;
9  begin
10   msg := 'Hello';  // 5 chars
11
12   Writeln(' msg: ', msg);
13
14   ptr := @msg[1];     // ptr = msg;
15
16   ptr^ := 'M';        // *ptr = 'M';
17   Inc(ptr);           // ptr++;
18
19   ptr^ := 'a';        // *ptr = 'a';
20
21   // Print the modified array as a string
22   Writeln('Modified msg: ', msg);
23 end.
24
```

chpoint.pas

Output:

```
 msg: Hello
Modified msg: Mallo
```



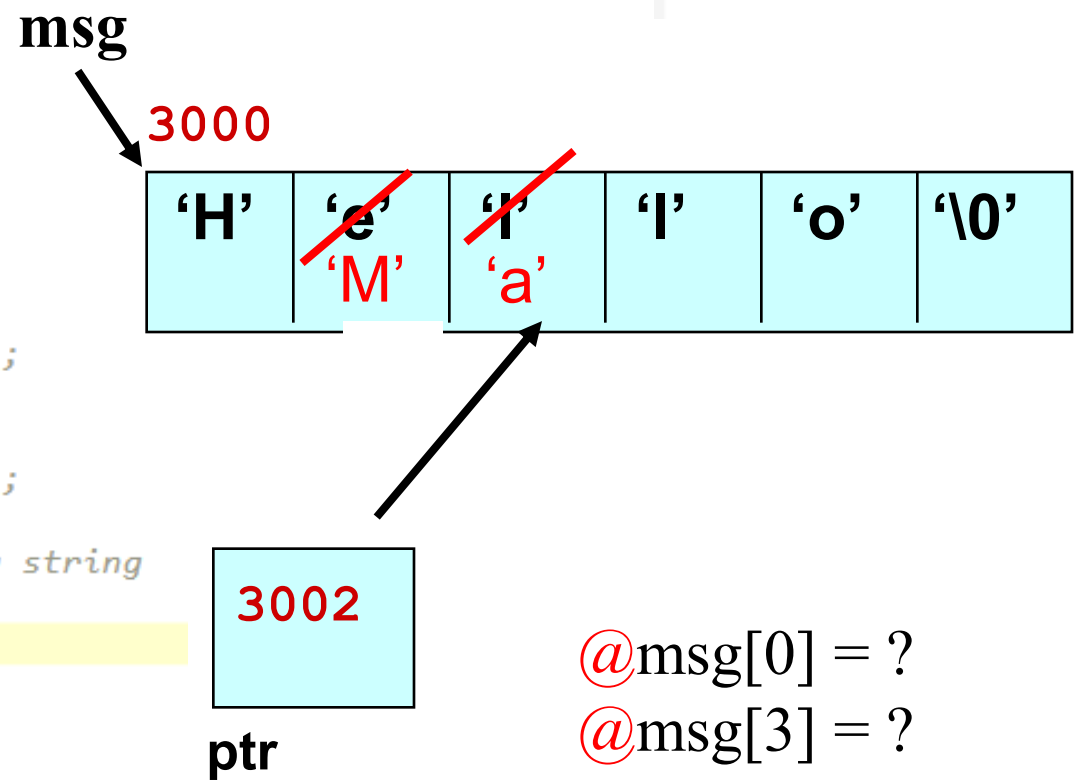
Pointer Arithmetic and Arrays (3.4)

```
1  program CharPointerExample;
2
3  uses
4    SysUtils;
5
6  var
7    msg: string;
8    ptr: ^Char;
9  begin
10   msg := 'Hello';  // 5 chars
11
12   Writeln(' msg: ', msg);
13
14   ptr := @msg[1];     // ptr = msg;
15
16   ptr^ := 'M';        // *ptr = 'M';
17   Inc(ptr);           // ptr++;
18
19   ptr^ := 'a';        // *ptr = 'a';
20
21   // Print the modified array as a string
22   Writeln('Modified msg: ', msg);
23 end.
24
```

chpoint.pas

Output:

```
 msg: Hello
Modified msg: Mallo
```



Array and pointer

P^[1] :=55

```
1  Program PtrArry;
2  type
3    TIntArray = array[1..5] of Integer;
4    PIntArray = ^TIntArray;
5
6
7  var
8    A : TIntArray;
9    pA: PIntArray;
10   i: Integer;
11
12 begin
13   A[1] := 55;
14   pA   := @A[2];
15
16   for i := 1 to 4 do
17   begin
18     pA^[i] := (i) * 10;
19   end;
20
21   for i := 1 to 5 do
22     writeln(A[i]);
23
24 end.
25
```

aryPtr.pas

Output:

```
55
10
20
30
40
```

Structure : Pointer

If a is a pointer to a struct, then to access the struct's members, we use the ^. operator as in a ^. x

```
1  program PointerRecordDemo;
2
3  type
4    TPoint = record
5      x : Integer;
6      y : Integer;
7    end;
8
9  var
10   p1 : TPoint;
11   p2 : ^TPoint;
12
13 begin
14   p1.x := 20;
15   p1.y := 30;
16
17   p2 := @p1;
18
19   writeln('p1.x = ', p1.x, ' , p1.y = ', p1.y);
20   writeln('p2^.x = ', p2^.x, ' , p2^.y = ', p2^.y);
21 end.
```

ptrRec.pas

Output:

```
p1.x = 20 , p1.y = 30
p2^.x = 20 , p2^.y = 30
```

Structure : Itself pointer

Step-1.Create Type of Pointer

```
Type
  TNode = record
    d :  integer;
    next : ^TNode ;
  end;
```

```
Type
  PNode = ^TNode ;
  TNode = record
    d :  integer;
    next : PNode;
  end;
```

Step-2.Create Variable

```
var
  data : TNode;
  Pdata: ^TNode;
```

```
var
  data : TNode;
  Pdata: PNode;
```

Structure : Itself pointer

วิธีการสร้าง

```
2
3  type
4    TNode = record
5      d     : Char;
6      x, y : Integer;
7      next : ^TNode;
8    end;
9    PNode = ^TNode;
10
11 var
12   a, b, c, d : TNode;
13   p          : PNode;
```

ไม่นิยม

```
2
3  type
4    PNode = ^TNode;
5    TNode = record
6      d     : Char;
7      x, y : Integer;
8      next : PNode;
9    end;
10
11 var
12   a, b, c, d : TNode;
13   p          : PNode;
```

นิยม

กรณีที่ Itselft Ponter: ไม่ได้เก็บข้อที่เป็นตำแหน่งตัวแปรใดๆ นิยมให้มีค่าเท่ากับ Nil หรือ Null (0)

a^.next := Nil

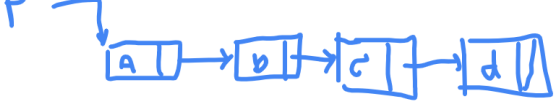
```pascal
program LinkedListDemo;

type
  PNode = ^TNode;
  TNode = record
    d    : Char;
    x, y : Integer;
    next : PNode;
  end;

var
  a, b, c, d : TNode;
  p          : PNode;

begin
  { initialize nodes }
  a.d := 'a'; a.x := 10; a.y := 20;
  b.d := 'b'; b.x := 11; b.y := 21;
  c.d := 'c'; c.x := 12; c.y := 22;
  d.d := 'd'; d.x := 13; d.y := 23;

  { link nodes }
  a.next := @b;
  b.next := @c;
  c.next := @d;
  d.next := nil;

  p := @a;

  writeln('p', #9, #9, 'd', #9, 'x', #9, 'y', #9, 'next');

  while p <> nil do
  begin
    writeln(
      PtrUInt(p), #9,
      p^.d, #9,
      p^.x, #9,
      p^.y, #9,
      PtrUInt(p^.next)
    );
    p := p^.next;
  end;
end.
```

Output:

```
p       d   x   y   next
4379696 a   10  20  4379712
4379712 b   11  21  4379728
4379728 c   12  22  4379744
4379744 d   13  23  0
```

simpleLL.pas

declare



addr	data	structure	variable
1000	'a'	d	
1001	10	x	a
1002	20	y	
1003	?	next	
1004	'b'	d	
1005	11	x	b
1006	21	y	
1007	?	next	
1008	'c'	d	
1009	12	x	c
1010	22	y	
1011	?	next	
1012	'd'	d	
1013	13	x	d
1014	23	y	
1015	?	next	
1016	?		p

a^.next =&b

b ^. next =&c

c ^. next =&d

d ^. next = Nil

p = @a

Execute

addr	data	structure	variable
1000	'a'	d	
1001	10	x	a
1002	20	y	
1003	1004	next	
1004	'b'	d	
1005	11	x	b
1006	21	y	
1007	1008	next	
1008	'c'	d	
1009	12	x	c
1010	22	y	
1011	1012	next	
1012	'd'	d	
1013	13	x	d
1014	23	y	
1015	NULL	next	
1016	1000		p

Phy vs Logical

addr	data	structure	variable
1000	'a'	d	
1001	10	x	a
1002	20	y	
1003	1004	next	
1004	'b'	d	
1005	11	x	b
1006	21	y	
1007	1008	next	
1008	'c'	d	
1009	12	x	c
1010	22	y	
1011	1012	next	
1012	'd'	d	
1013	13	x	d
1014	23	y	
1015	NULL	next	
1016	1000		p


```
11  var
12    a, b, c, d : TNode;
13    p          : PNode;
14
15  begin
16    { initialize nodes }
17    a.d := 'a'; a.x := 10; a.y := 20;
18    b.d := 'b'; b.x := 11; b.y := 21;
19    c.d := 'c'; c.x := 12; c.y := 22;
20    d.d := 'd'; d.x := 13; d.y := 23;
21
22    { link nodes }
23    a.next := @b;
24    b.next := @c;
25    c.next := @d;
26    d.next := nil;
27
28    p := @a;
29
30    writeln('p', #9, #9, 'd', #9, 'x', #9, 'y', #9, 'next');
31
32    while p <> nil do
33    begin
34      writeln(
35        PtrUInt(p), #9,
36        p^.d, #9,
37        p^.x, #9,
38        p^.y, #9,
39        PtrUInt(p^.next)
40      );
41      p := p^.next;
42    end;
43  end.
```

Pointer Parameter Passing

- Using pass by value technique
- The result is the same as variable pass by refer.

Pointer parameter : (Pointer) pass by value

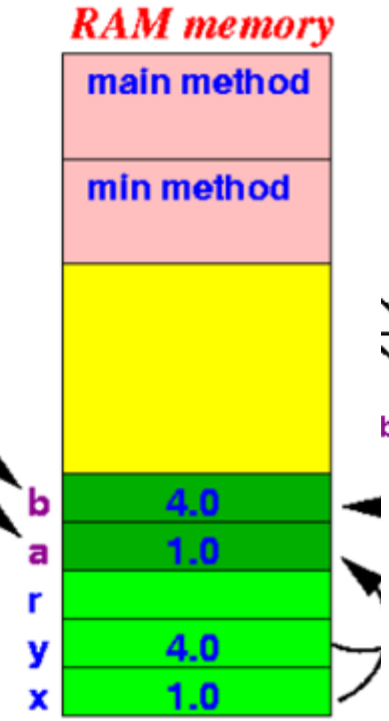
x,y:variable

x,y:pointer variable

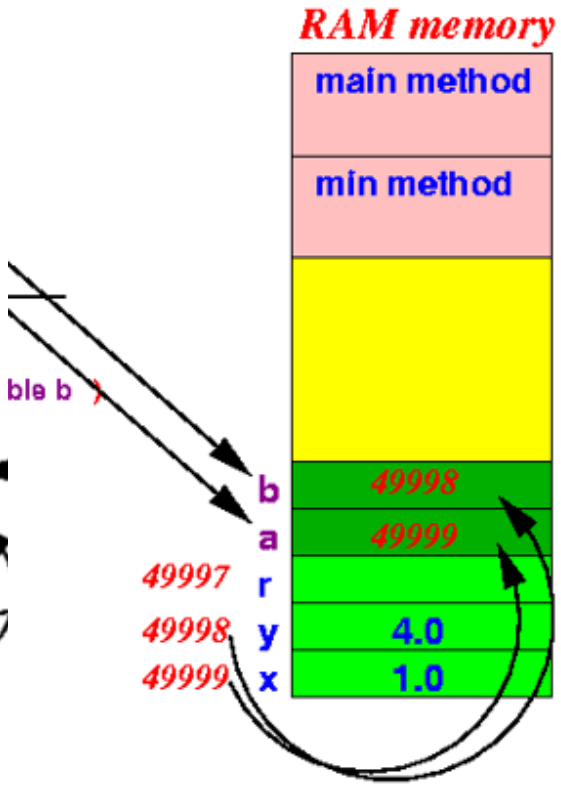
```
public class MyProgram
{
   public  static  void  main( String[] args )
   {
      double x = 1.0, y = 4.0;
      double r;
      r =  ToolBox.min( x , y );
      ....
   }
}
```

```
public class ToolBox
{
   public static double  min ( double a, double b )
   {
      double m = 0;
      if ( a < b )
      {
         m = a;
      }
      else
      {
         m = b;
      }
      return(m);
   }
}
```

Pass-by-value:
2. copy value of actual parameter to formal parameter



Pass-by-reference:
2. copy reference of actual parameter to formal parameter



การเขียนโปรแกรมโดยใช้ Pointer ร่วมกับ subroutine ในภาษา PASCAL

Step-1.Create Type of Pointer

```
Type
 TPoint = record
   x : Integer;
   y : Integer;
 end;
 PPoint = ^TPoint;
 Pint    = ^Integer
```

Step-2.Create Function

```
Procedure Func( x: PPoint  ; y Pint );
Begin

End;
```

Step-3. Caller

```
var     a1:TPoint;   a2:PPoint
        b1:Integer ;  b2: Pint;
Func( @a1, @b1);
a2 := @a1;   b2 :=@b1;
Func( a2, b2);
```

```
type
  PInt = ^Integer;

procedure PointerPassByValue(P: Pint);
begin
  writeln( #9,'A1.P= ', p^ );
  P^ := 10+P^;
  writeln( #9,'A2.P= ',p^ );
end;

procedure PassByRef( var P: Integer);
begin
  writeln( #9,#9,'B1.P= ',p);
  P := 100+P;
  writeln( #9,#9,'B2.P= ',p);
end;

var
   A : Integer;
  pA : PInt;
```

```
C1.A= 1
        A1.P= 1
        A2.P= 11
C2.A= 11
                B1.P= 11
                B2.P= 111
C3.P= 111

C1.P= 2
        A1.P= 2
        A2.P= 12
C2.P= 12
                B1.P= 12
                B2.P= 112
C3.P= 112
```

ParmPtrVar.pas

```
begin
  A := 1;
  writeln( 'C1.A= ',A);

  PointerPassByValue(@A);
  writeln( 'C2.A= ',A);

  PassByRef(A);
  writeln( 'C3.P= ',A);

  //PassByRef(@A);
  //writeln( 'C3.P= ',A);

  writeln;
  pA := @A;
  pA^ := 2;
  writeln( 'C1.P= ',A);

  PointerPassByValue(pA);
  writeln( 'C2.P= ',A);

  PassByRef( pA^ );
  writeln( 'C3.P= ',A);

  //PointerPassByValue( @pA );
  //PassByRef( @pA );
  //writeln( 'C4.P= ',A);

end.
```

Pointer parameter : (Pointer) pass by value

parmArryPtr.pas

```pascal
Program PtrArry;
type
  TIntArray = array[1..5] of Integer;
  PIntArray = ^TIntArray;

procedure writeArry(P: PIntArray);
var
  i: Integer;
begin
  for i := 1 to 5 do
    writeln(P^[i]);
  writeln('---');
end;

procedure FillArray(P: PIntArray);
var
  i: Integer;
begin
  for i := 1 to 5 do
    P^[i] := P^[i] + 100;
end;

procedure FillArrayRef(var P: TIntArray);
var
  i: Integer;
begin
  for i := 1 to 5 do
    P[i] := P[i] + 1000;
end;
```

```pascal
var
  A : TIntArray;
  pA: PIntArray;
  i: Integer;

begin
  A[1] := 55;
  pA   := @A[2];
  for i := 1 to 4 do
  begin
    pA^[i] := (i) * 10;
  end;

  writeArry(@A);

  FillArray(@A);

  writeArry(@A);

  FillArrayRef(A);
  writeArry(@A);

end.
```


Pointer parameter : pass by value (Pointer)

ptrFunc.pas

```
1  program ptrFunc;
2  type
3    TPoint = record
4      x : Integer;
5      y : Integer;
6    end;
7    PTPoint = ^TPoint;
8
9  procedure Output(var p1 : TPoint ; p2 : PTPoint );
10 begin
11   writeln('1.p1.x = ', p1.x, ' , p1.y = ', p1.y);
12   writeln('@p1 = ', HexStr(@p1) ) ;
13   writeln('1.p2^.x = ', p2^.x, ' , p2^.y = ', p2^.y);
14   writeln('2.p2^.x = ', p2^.x, ' , p2^.y = ', p2^.y);
15   writeln('p2 = ', HexStr(p2 ) ) ;
16   writeln('@p2 = ', HexStr(@p2 ) );
17   writeln;
18 end;
19
20 var
21   x1 : TPoint;
22   x2 : TPoint;
23   px2 : PTPoint;
24 begin
25   x1.x := 10;
26   x1.y := 20;
27   writeln('@x1 = ', HexStr(@x1) ) ;
28   Output(x1, @x1);
29
30   x2.x := 100;
31   x2.y := 200;
32   writeln('@x2 = ', HexStr(@x2) ) ;
33   Output(x2, @x2);
34
35   px2 := @x2;
36   writeln('@px2 = ', HexStr(@px2) ) ;
37   writeln('px2 = ', HexStr(px2) ) ;
38   Output( px2^ , px2);
39
40 end.
```

```
procedure Output(var p1 :
Free Pascal Compiler version 3.
Copyright (c) 1993-2021 by Flor
Target OS: Linux for x86-64
Compiling main.pas
Linking a.out
42 lines compiled, 0.0 sec
@x1 = 0000000000042E830
1.p1.x = 10 , p1.y = 20
@p1 = 0000000000042E830
1.p2^.x = 10 , p2^.y = 20
2.p2^.x = 10 , p2^.y = 20
p2 = 0000000000042E830
@p2 = 00007FFD54303260

@x2 = 0000000000042E840
1.p1.x = 100 , p1.y = 200
@p1 = 0000000000042E840
1.p2^.x = 100 , p2^.y = 200
2.p2^.x = 100 , p2^.y = 200
p2 = 0000000000042E840
@p2 = 00007FFD54303260

@px2 = 0000000000042E850
px2 = 0000000000042E840
1.p1.x = 100 , p1.y = 200
@p1 = 0000000000042E840
1.p2^.x = 100 , p2^.y = 200
2.p2^.x = 100 , p2^.y = 200
p2 = 0000000000042E840
@p2 = 00007FFD54303260
```

```
type
  PNode = ^TNode;
  TNode = record
    d    : Char;
    x, y : Integer;
    next : PNode;
  end;

var
  a, b, c, d : TNode;
  p          : PNode;

begin
  { initialize nodes }
  a.d := 'a'; a.x := 10; a.y := 20;
  b.d := 'b'; b.x := 11; b.y := 21;
  c.d := 'c'; c.x := 12; c.y := 22;
  d.d := 'd'; d.x := 13; d.y := 23;

  { link nodes }
  a.next := @b;
  b.next := @c;
  c.next := @d;
  d.next := nil;

  p := @a;

  writeln('p', #9, #9, 'd', #9, 'x', #9, 'y', #9, 'next');

  while p <> nil do
  begin
    writeln(
      PtrUInt(p), #9,
      p^.d, #9,
      p^.x, #9,
      p^.y, #9,
      PtrUInt(p^.next)
    );
    p := p^.next;
  end;
end.
```

Convert to subroutine

simpleLL.pas

ptrFuncPrt.pas

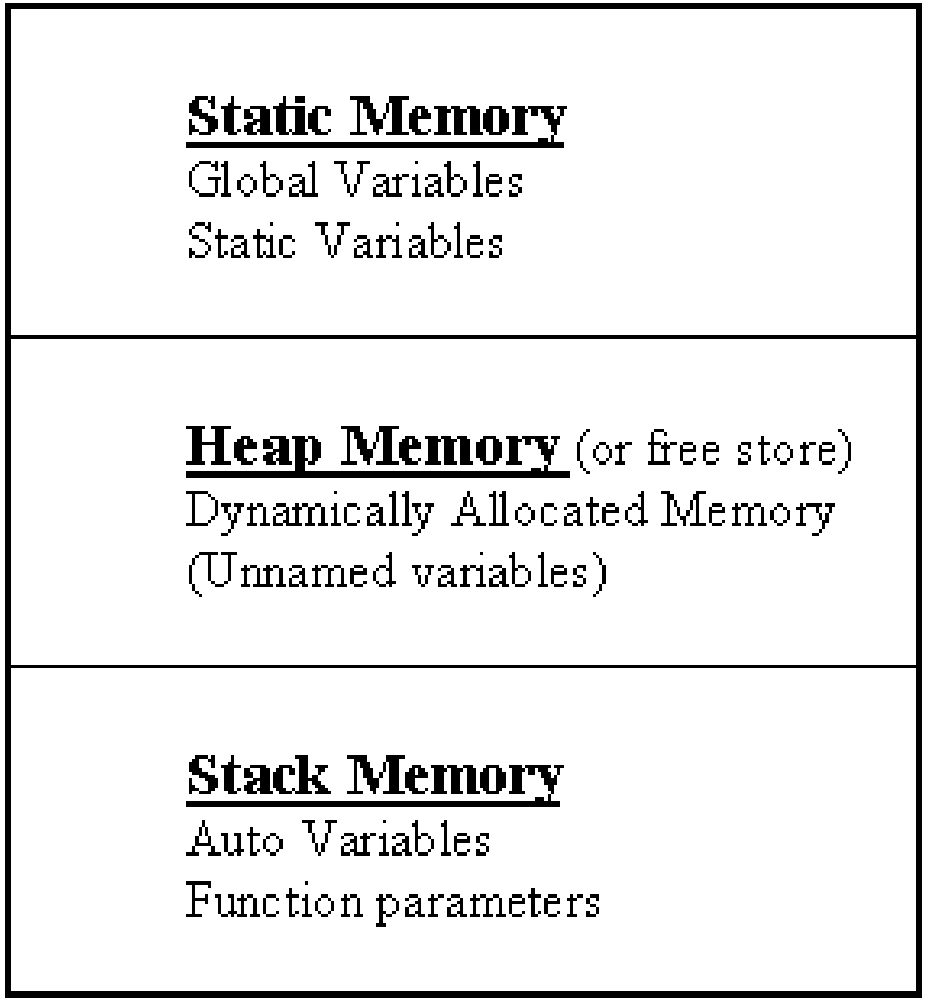
```
1  program LinkedListDemo;
2
3  type
4    PNode = ^TNode;
5    TNode = record
6      d    : Char;
7      x, y : Integer;
8      next : PNode;
9    end;
10
11 procedure Print(p:PNode);
12 begin
13   writeln( '1.p= ', HexStr(p));
14
15   writeln('p', #9, #9, 'd', #9, 'x'
16     , #9, 'y', #9, 'next');
17   while p <> nil do
18   begin
19     writeln(
20       PtrUInt(p), #9,
21       p^.d, #9,
22       p^.x, #9,
23       p^.y, #9,
24       PtrUInt(p^.next)
25     );
26     p := p^.next;
27   end;
28
29   writeln( '2.p= ', HexStr(p));
30
31 end;
```

```
33
34 var
35   a, b, c, d : TNode;
36   p          : PNode;
37
38 begin
39   { initialize nodes }
40   a.d := 'a'; a.x := 10; a.y := 20;
41   b.d := 'b'; b.x := 11; b.y := 21;
42   c.d := 'c'; c.x := 12; c.y := 22;
43   d.d := 'd'; d.x := 13; d.y := 23;
44
45   { link nodes }
46   a.next := @b;
47   b.next := @c;
48   c.next := @d;
49   d.next := nil;
50
51   p := @a;
52
53   writeln( '4.p= ', HexStr(p));
54   Print(p);
55   writeln( '5.p= ', HexStr(p));
56
57 end.
```

Dynamic Memory Allocation

In Pascal three types of memory are used by programs:

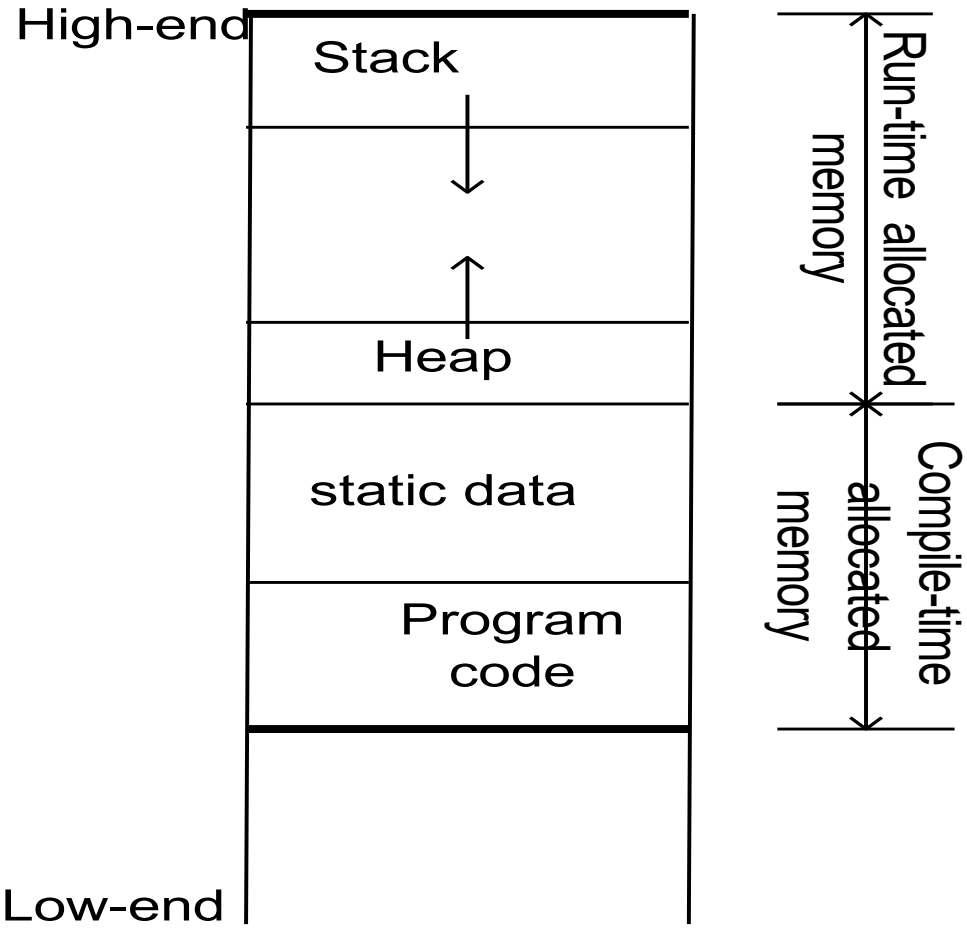
- **Static memory** - where global and static variables live
- **Heap memory** - dynamically allocated at execution time
  - "managed" memory accessed using pointers
- **Stack memory** - used by automatic variables



3 Kinds of Program Data

- STATIC DATA: Allocated at compiler time
- DYNAMIC DATA: explicitly allocated and deallocated during program execution by Pascal instructions written by programmer using operators **`new`** and **`dispose`**
- AUTOMATIC DATA: automatically created at function entry, resides in activation frame of the function, and is destroyed when returning from function

Dynamic Memory Allocation Diagram



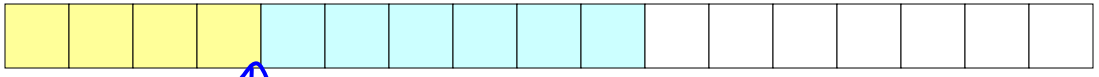
Dynamic Memory Allocation

- *In Pascal*, functions such as **new()** are used to dynamically allocate memory from the **Heap** and **depose()**
- **new** is used to allocate memory during execution time
  - returns a pointer to the address where the object is to be stored
  - always returns a pointer to the type that follows the **new**

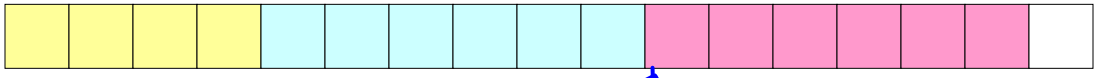
Allocation Examples

```
p1 = new(4)
```


```
p2 = new(5)
```




```
p3 = new(6)
```

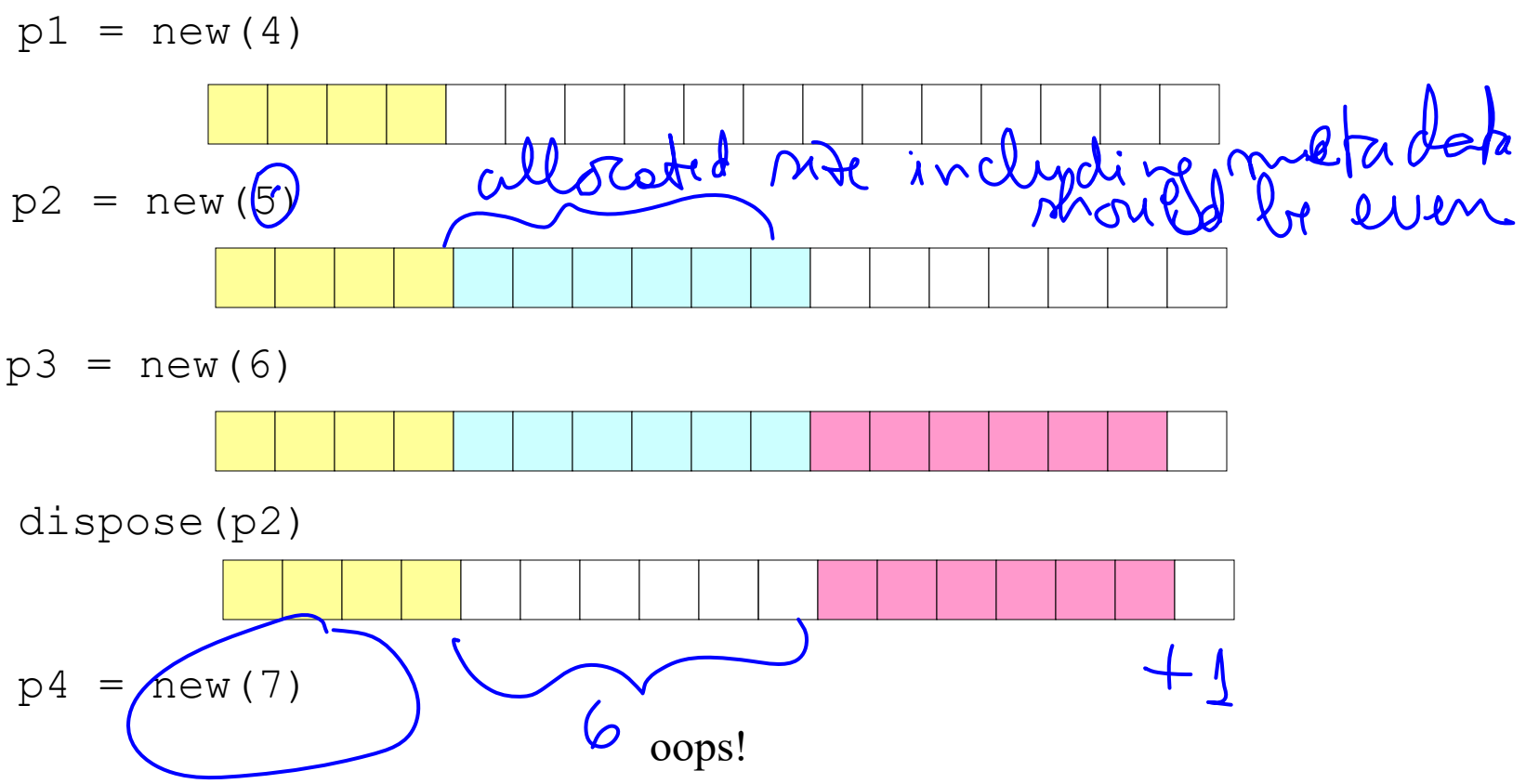



```
dispose(p2)
```


```
p4 = new(2)
```

External Fragmentation

Occurs when there is enough aggregate heap memory, but no single free block is large enough



External fragmentation depends on the pattern of *future* requests, and thus is difficult to measure.

Example

tempNew.pas

```
1  program AllocTmpl;
2
3  type
4    { ---------- array of Integer ---------- }
5    TIntArray = array[1..5] of Integer;
6    PIntArray = ^TIntArray;
7
8
9  var
10   pIntArr: PIntArray;
11 begin
12   { ===== allocate ===== }
13   New(pIntArr); //array of int
14
15   if(pIntArr <> nil ) then
16   begin
17     pIntArr^[3] := 3;
18     writeln(pIntArr^[3]);
19
20     { ===== free ===== }
21     Dispose(pIntArr);
22   end;
23
24 end.
```

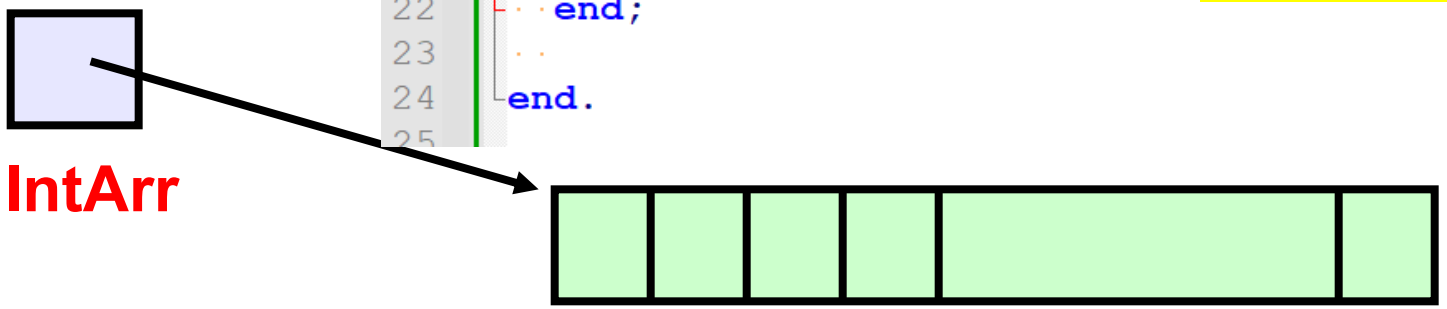
1.กำหนด datatype ที่ต้องการจอง

2.กำหนด datatype ของ pointer

3.กำหนด ตัวแปร ของ pointer

4.จอง memory

5.คืนพื้นที่ Memory



5*4 bytes of space

Example-1

exNewDisp.pas

```pascal
program AllocateArrayAndStruct;

type
  { ---------- array of Integer ---------- }
  TIntArray = array[1..5] of Integer;
  PIntArray = ^TIntArray;

  { ---------- record ---------- }
  TStudent = record
    id: Integer;
    score: Integer;
  end;

  { ---------- array of record ---------- }
  TStudentArray = array[1..3] of TStudent;
  PStudentArray = ^TStudentArray;

  PStudent = ^TStudent;

var
  IntArr: PIntArray;
  StuArr: PStudentArray;
  pStu: PStudent;
  i: Integer;
begin
  { ===== allocate ===== }
  New(IntArr); //array of int
  New(StuArr); //array of TStudent
  New(pStu);   //TStudent

  { ===== use: array of Integer ===== }
  for i := 1 to 5 do
    IntArr^[i] := i * 10;

  { ===== use: array of record ===== }
  for i := 1 to 3 do
  begin
    StuArr^[i].id := i;
    StuArr^[i].score := i * 25;
  end;

  { ===== example access ===== }
  WriteLn('IntArr[3] = ', IntArr^[3]);
  WriteLn('Student 2: id=', StuArr^[2].id,
          ' score=', StuArr^[2].score);

  pStu^.id    := 1000;
  pStu^.score := 10000;
  WriteLn('pStu^.id=', pStu^.id,
  ' pStu^.score=', pStu^.score);


  { ===== free ===== }
  Dispose(IntArr);
  Dispose(StuArr);
  Dispose(pStu);
end.
```

Example-2

กรณี allocate array

exAlloc.pas

NOTE:การ allocate ข้อมูลที่เป็น array ควรต้องประเมินว่า max ของ data ที่ระบบต้องมีขนาดเท่าไร แล้วจึงสร้าง array Type เพื่อเก็บข้อมูล

```
1  program AverageHeight_NewDispose;
2
3  type
4    TFloatArray = array[1..15] of real;
5    PFloatArray = ^TFloatArray;
6
7  var
8    i, N: Integer;
9    height: PFloatArray;
10   sum, avg: real;
11
12 begin
13   sum := 0;
14
15   write('Input number of students (1-15) : ');
16   readln(N);
17
18   { allocate N elements }
19   new(height);
20
21   for i := 1 to N do
22   begin
23     writeln('Input heights for ', i, ' students');
24     readln(height^[i]);
25   end;
26
27   for i := 1 to N do
28     sum := sum + height^[i];
29
30   avg := sum / N;
31
32   writeln('Average height = ', avg:0:2);
33
34   { deallocate memory }
35   dispose(height);
36 end.
37
```

Ex1

main.pas

```pascal
program AverageDifferenceLoop;
var
  score: array[1..5] of real;
  avg, dif, sum: real;
  i:integer;
begin
  sum := 0;
  { --- Input 5 scores --- }
  for i := 1 to 5 do
  begin
    readln(score[i]);
    sum := sum + score[i];
  end;

  avg := sum / 5;

  writeln('average = ', avg:0:2);

  { --- Show differences --- }
  for i := 1 to 5 do
  begin
     dif := score[i] - avg;
     writeln(dif:0:2);
  end;
end.
```

จงเขียนโปรแกรม อ่านคะแนนจากแป้นพิมพ์จำนวน **5** ค่า แล้วนำมาคำนวณค่าเฉลี่ย และหาผลต่างของแต่ละค่ากับค่าเฉลี่ย

Convert to static to dynamic

Ex2 (1)

Convert to static to dynamic

StruEmpAvg.pas

สมมุติว่าข้อมูล อาจารย์คณะวิทยาศาสตร์มี 150 ดังนี้

เพศ (sex) : ชาย = 1 , หญิง = 2

อายุ (age) : ? ปี

สถานภาพสมรส (status) : โสด = 1 , แต่งงาน = 2 , อื่นๆ = 3

ปริญญาสูง (degree) : ตรี = 1 , โท = 2 , เอก = 3

จำนวนปีที่สอน (experience) : ? ปี

จงเขียน

1. โปรแกรมรับข้อมูลของอาจารย์คณะวิทยาศาสตร์จำนวน 150 คน
2. หาจำนวนปีเฉลี่ยของ อายุ และ จำนวนปีทีสอน จำนวนคนที่มีค่าอายุและปรสบการณ์มากกว่าค่าเฉลี่ย

```
AVEREAGE  AGE                    =      xx.xx  Years
NUMBER OF AGE ABOVE AVEAGE  =       XX
AVEREAGE  Experience             =      XX.XX Years
NUMBER OF AGE ABOVE AVEAGE  =       XX
```

Ex2 (2)

StruEmpAvg.pas

main.pas

```
1  program EmployeeAverage;
2  const numPerson = 3; //ค่าคงที่
3  type
4    Employee = record
5      sex: Integer;
6      age: Integer;
7      stat: Integer;
8      degr: Integer;
9      tyear: Integer;
10   end;
11
12 var
13   person: array[1..numPerson] of Employee;
14   i: Integer;
15   avg_age, avg_expr: Real;
16   numAgeAbove, numYearAbove: Integer;
17
18 begin
19   { Input data for first 2 employees }
20   for i := 1 to numPerson do
21   begin
22     Write('sex(1-2) = ');   ReadLn(person[i].sex);
23     Write('age = ');   ReadLn(person[i].age);
24     Write('stat (1-3) = ');  ReadLn(person[i].stat);
25     Write('degr(1-3) = ');  ReadLn(person[i].degr);
26     Write('tyear = '); ReadLn(person[i].tyear);
27   end;
28
29   avg_age := 0;
30   avg_expr := 0;
```

Ex2 (3)

StruEmpAvg.pas

```
29    avg_age := 0;
30    avg_expr := 0;
31
32    { Compute sums }
33    for i := 1 to numPerson do
34    begin
35      avg_age := avg_age + person[i].age;
36      avg_expr := avg_expr + person[i].tyear;
37    end;
38
39    avg_age := avg_age / numPerson;
40    avg_expr := avg_expr / numPerson;
41
42    numAgeAbove := 0;
43    numYearAbove := 0;
44
45    { Count how many are above average }
46    for i := 1 to numPerson do
47    begin
48      if person[i].age > avg_age then
49        numAgeAbove := numAgeAbove + 1;
50
51      if person[i].tyear > avg_expr then
52        numYearAbove := numYearAbove + 1;
53    end;
54
55    { Output results }
56    WriteLn('AVG AGE  = ', avg_age:0:2);
57    WriteLn('NUMBER OF AGE ABOVE AVERAGE = ', numAgeAbove);
58    WriteLn('AVG EXPR = ', avg_expr:0:2);
59    WriteLn('NUMBER OF YEAR ABOVE AVERAGE = ', numYearAbove);
60  end.
```

Dynamic กับ Subroutine

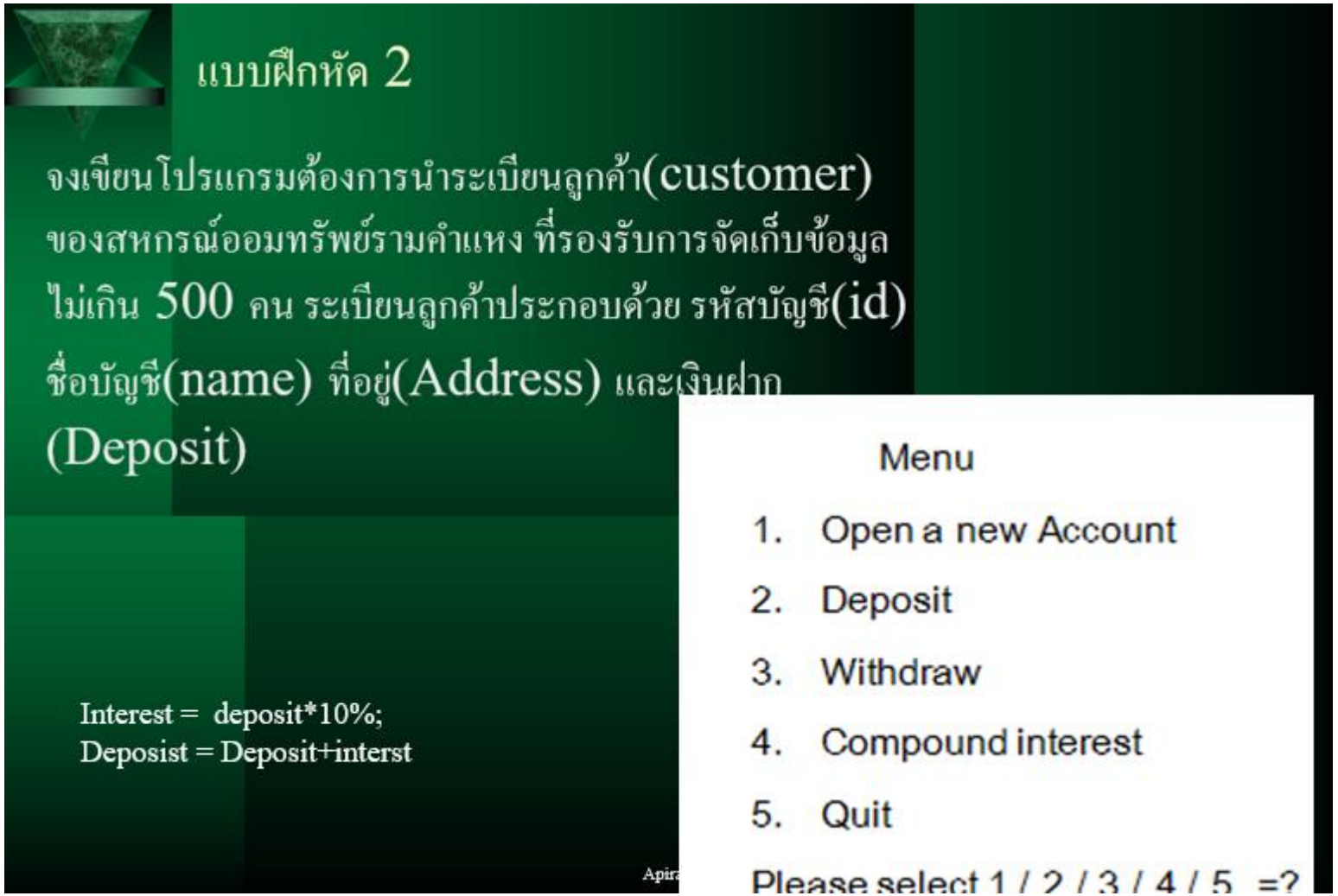
main.pas

```
1  type
2    ArryInt = array[1..5] of Integer;
3    PArryInt = ^ArryInt;
4
5  function newArray():PArryInt;
6  var p:PArryInt;
7     i:Integer;
8  begin
9   new(p);
10  for i:=1 to 5 do
11    p^[i] := i;
12  exit(p);
13 end;
14
15 procedure print( p:PArryInt );
16 var   i:Integer;
17 begin
18  for i:=1 to 5 do
19    writeln( 'p^[',i,'] =', i);
20 end;
21
22 procedure delArray( p:PArryInt );
23 begin
24  dispose(p);
25  p := nil;
26 end;
```

```
34 var
35   pA : PArryInt;
36   pB : PArryInt;
37
38 begin
39   pA := newArray();
40   writeln( 'pA = ', HexStr(pA));
41   print(pA);
42   delArray(pA);
43
44   writeln( 'pA = ', HexStr(pA));
45
46   pB := newArray();
47   writeln( 'pB = ', HexStr(pA));
48   print(pB);
49   delArrayRef(pB);
50
51  writeln( 'pB = ', HexStr(pB));
52
53 end.
```

BackAccV2.pas

Ex2 (3)



Ex2 (4)

BackAccV2.pas

main.pas

```
1  program BankAccount;
2
3  type
4    Account = record
5      id: string[10];
6      name: string[100];
7      addr: string[100];
8      deposit: real;
9    end;
10
11   AccountDataType = array[1..500] of Account;
12
13 function getChoice:integer;
14 var
15   choice:integer;
16 begin
17     writeln;
18     writeln('1. Open Acc');
19     writeln('2. Deposit Acc');
20     writeln('3. Withdraw Acc');
21     writeln('4. Compute Interest');
22     writeln('5. Exit');
23     write('Enter choice: ');
24     readln(choice);
25     exit(choice);
26 end;
```

Ex2 (5)

BackAccV2.pas

```
27
28  Procedure OpenAcc( var cust:AccountDataType ; var n:integer );
29  var
30        id: string[10];
31        bfound: boolean;
32        i:integer;
33  begin
34      writeln;
35      writeln('Do Open Acc');
36      write('id Acc =? ');
37      readln(id);
38      bfound := False;
39
40      for i := 1 to n do
41      begin
42      if cust[i].id = id then
43      begin
44        bfound := True;
45        break;
46      end; // end if
47      end; //end for
48
49      if  bfound = False then
50      begin
51          n := n + 1;
52          cust[n].id := id;
53          write('name Acc =? ');
54          readln(cust[n].name);
55          write('addr Acc =? ');
56          readln(cust[n].addr);
57          write('deposit Acc =? ');
58          readln(cust[n].deposit);
59      end
60      else
61          writeln('id Acc found = ', id);
62  end;
```

Ex2 (6)

BackAccV2.pas

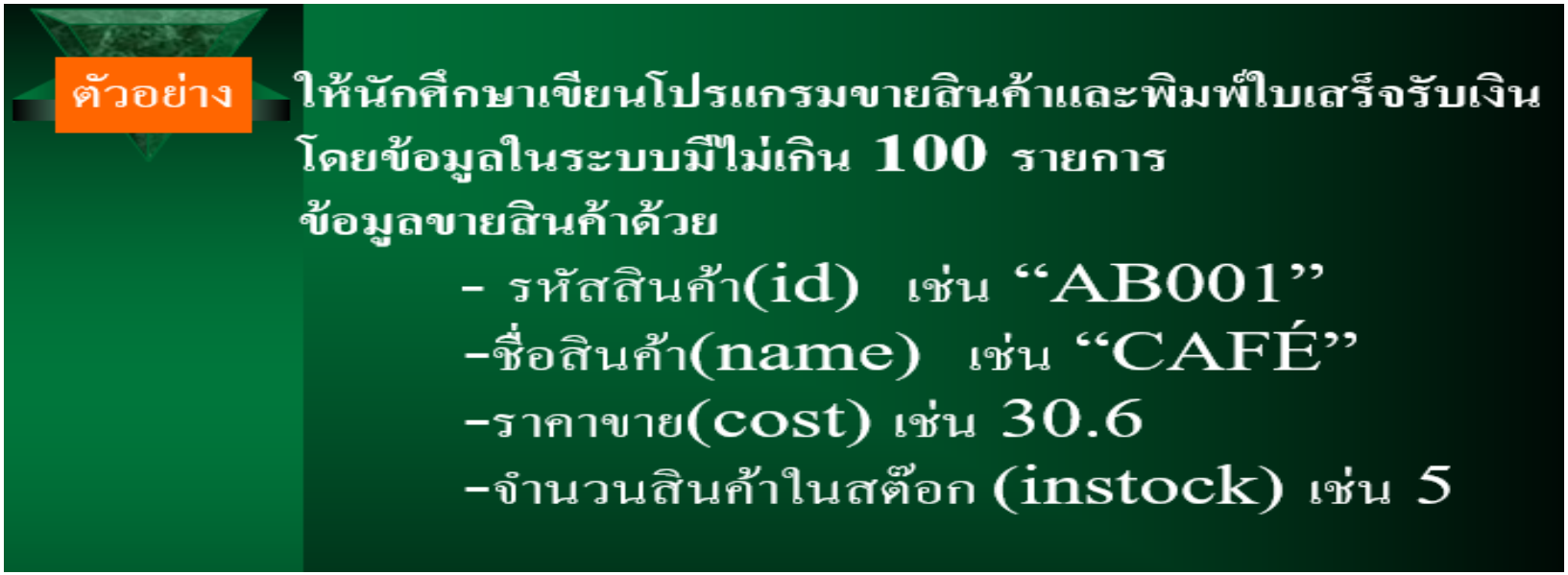
```
64  Procedure Deposit( var cust:AccountDataType ; var n:integer );
65  var
66        id: string[10];
67        bfound: boolean;
68        deposit: real;
69        i:integer;
70  begin
71    writeln;
72    writeln('Do Deposit Acc');
73    write('id Acc =? ');
74    readln(id);
75
76    bfound := False;
77    for i := 1 to n do
78    begin
79      if cust[i].id = id then
80      begin
81        bfound := True;
82        break;
83      end; //end if
84    end; // end for
85
86    if bfound = False then
87      writeln('id Acc not found = ', id)
88    else
89     begin
90        writeln('Before deposit Acc = ', cust[i].deposit:0:2);
91        write('New deposit Acc =? ');
92        readln(deposit);
93        cust[i].deposit := cust[i].deposit + deposit;
94        writeln('After deposit Acc = ', cust[i].deposit:0:2);
95     end;
96  end; //case 2
```

BackAccV2.pas

```
 99 var
100   cust: AccountDataType;
101   n, choice, i: integer;
102
103
104 begin
105   n := 0;
106   repeat
107     choice :=  getChoice();
108     case choice of
109       1: OpenAcc( cust , n );
110       2: Deposit( cust ,n );
111       3:
112         begin
113           writeln('Do Withdraw Acc');
114         end;
115       4:
116         begin
117           writeln('Do Interest Acc');
118         end;
119     end;
120
121   until choice = 5;
122 end.
```

Ex3 (1)

Rewrite the code using function memory allocation



ep30_labcode.pas

Ex3 (2)

```
        Menu
1.      Add item to Cart
2.      Sell Product
3.      Check Stock
4.      End Program
Please select 1 / 2 / 3/ 4 =?
```

กด 1

```
Input your product data:
id :  AB001
name : CAFE
Cost :  30.6
Stock  : 5
```

กด 2

```
Input your product id: AB001
Input your sell volume : 2
name : CAFE
Cost :  30.6
stock : 3
Pay : 61.2 bath
```

```
        Menu
1.      Add item to Cart
2.      Sell Product
3.      Check Stock
4.      End Program
Please select 1 / 2 / 3/ 4 =?
```

กด 3

```
product List:
Id          name    cost  stock
- - - - - - - - - - - - - - - - - -
AB001  CAFÉ   30.6    3
```

กด 4

exit from program

ep30_labcode.pas

Ex3 (4)

1

```
ptrFuncPrt.pas | ParmPtrVar.pas
1  program recordDeclare;
2  type
3      Stock = record
4      id: String;
5      name: string[50];
6      cost: real;
7      instock: Integer;
8      end;
9      aryStock = array [1..100] of Stock;
10
11 var
12     myStock:aryStock;
13     n:Integer;
14     
15 procedure  AddStock( id:string ; name:string  ;
16                    cost:real ; stock:Integer );
17 begin
18     n := n+1;
19     myStock[n].id := id;
20     myStock[n].name := name;
21     myStock[n].instock := stock;
22     myStock[n].cost := cost;
23 end;
24
25 procedure  updateStock( nFound:Integer ; cost:real );
26 begin
27     //myStock[nFound].id := id;
28     //myStock[nFound].name := name;
29     //myStock[nFound].stock := stock;
30     writeln( ' old cost ' , myStock[nFound].cost:0:2);
31     myStock[nFound].cost := cost;
32     writeln( ' update cost ' , myStock[nFound].cost:0:2);
33 end;
```

ep30_labcode.pas

Ex3 (5)

2

```
35  function FindStockId(id:string):Integer;
36  var
37   i:Integer;
38  begin
39    for i:= 1 to n  do
40    begin
41      if myStock[i].id = id then
42      begin
43        Exit( i);
44      end;
45    end;
46    Exit(0);
47  end;
```

```
49  procedure addItem();
50  var
51    nFound:Integer;
52    id:string;
53      name: string[50];
54      cost: real;
55      stock: Integer;
56  begin
57      writeln('Add itm');
58      //id := readId();
59      write('id = ' ); readln( id );
60      nFound := FindStockId(id) ;
61      if nFound <= 0 then
62        begin
63          write('name = ' ); readln( name );
64          write('cost = ' ); readln( cost );
65          write('stock = ' );readln( stock );
66          AddStock( id,name,cost,stock );
67        end
68      else
69        begin
70            writeln('new cost = ' ); readln( cost );
71            updateStock( nFound, cost );
72        end;
73
74  end;
```

ep30_labcode.pas

Ex3 (6)

3

```
76  procedure sellItem();
77  var
78    nFound:Integer;
79    id:string;
80    volumn: Integer;
81
82  begin
83      writeln('sell itm');
84      write('id = ');readln( id );
85      nFound := FindStockId(id) ;
86      if nFound <= 0 then
87      begin
88          write('stock not found');
89      end
90      else
91          begin
92              writeln('get volumn = ');readln( volumn );
93              writeln( ' old volumn ' , myStock[nFound].instock);
94              if( myStock[nFound].instock - volumn >= 0 ) then
95              begin
96                  myStock[nFound].instock := myStock[nFound].instock - volumn;
97                  writeln( ' update volumn ' , myStock[nFound].instock);
98              end
99              else
100                 writeln( ' not enougth volumn' );
101         end;
102 end;
```

ep30_labcode.pas

Ex3 (7)

4

```
105  procedure  checkItem();
106  begin
107      writeln('check Item');
108  end;
109
110  function getMenu():Integer;
111  var
112    ch:Integer;
113  begin
114   writeln('1.Add item');
115   writeln('2.Sell item');
116   writeln('3.Check item');
117   writeln('4.Exit item');
118   readln( ch);
119   exit( ch );
120  end;
```

```
122  var
123    ch:Integer;
124
125  begin
126    n:= 0;
127
128    ch := getMenu();
129    while( ch <> 4) do
130    begin
131      if ch = 1 then
132      begin
133        addItem();
134      end
135      else
136      if ch = 2 then
137      begin
138        sellItem();
139      end
140      else if ch = 3 then
141      begin
142        checkItem();
143      end;
144     ch := getMenu();
145    end;
146
147  end.
```

Ex4 (1)

1.Convert a,b,c,d to be pointer
2.New/dispose

```
program LinkedListDemo;

type
  PNode = ^TNode;
  TNode = record
    d    : Char;
    x, y : Integer;
    next : PNode;
  end;

var
  a, b, c, d : TNode;
  p          : PNode;

begin
  { initialize nodes }
  a.d := 'a'; a.x := 10; a.y := 20;
  b.d := 'b'; b.x := 11; b.y := 21;
  c.d := 'c'; c.x := 12; c.y := 22;
  d.d := 'd'; d.x := 13; d.y := 23;

  { link nodes }
  a.next := @b;
  b.next := @c;
  c.next := @d;
  d.next := nil;

  p := @a;

  writeln('p', #9, #9, 'd', #9, 'x', #9, 'y', #9, 'next');

  while p <> nil do
  begin
    writeln(
      PtrUInt(p), #9,
      p^.d, #9,
      p^.x, #9,
      p^.y, #9,
      PtrUInt(p^.next)
    );
    p := p^.next;
  end;
end.
```

Ex4 (2)

```
program LinkedListDemo;
type
  PNode = ^TNode;
  TNode = record
    d    : Char;
    x, y : Integer;
    next : PNode;
  end;
var
  a, b, c, d : PNode; //TNode;
  p          : PNode;

begin
  { initialize nodes }
  new(a);
  a^.d := 'a'; a^.x := 10; a^.y := 20;

  new(b);
  b^.d := 'b'; b^.x := 11; b^.y := 21;

  new(c);
  c^.d := 'c'; c^.x := 12; c^.y := 22;

  new(d);
  d^.d := 'd'; d^.x := 13; d^.y := 23;

  { link nodes }
  a^.next := b;
  b^.next := c;
  c^.next := d;
  d^.next := nil;

  p := a;  //p := @a;

  writeln('p',#9,#9, #9, #9,#9, 'd',
          #9, 'x', #9, 'y', #9, 'next');

  while p <> nil do
  begin
    writeln(
      HexStr( p), #9,
      p^.d, #9,
      p^.x, #9,
      p^.y, #9,
      HexStr( p^.next )
    );
    p := p^.next;
  end;

  dispose(a);
  dispose(b);
  dispose(c);
  dispose(d);
end.
```

ให้สร้าง

Ex4 (3)

1.function
newTNODE(x,y)

2.Function
deleteTNODE(p)

```
1  program LinkedListDemo;
2  type
3    PNode = ^TNode;
4    TNode = record
5      d    : Char;
6      x, y : Integer;
7      next : PNode;
8    end;
9  var
10   a, b, c, d : PNode; //TNode;
11   p          : PNode;
12
13 begin
14   { initialize nodes }
15   new(a);
16   a^.d := 'a'; a^.x := 10; a^.y := 20;
17
18   new(b);
19   b^.d := 'b'; b^.x := 11; b^.y := 21;
20
21   new(c);
22   c^.d := 'c'; c^.x := 12; c^.y := 22;
23
24   new(d);
25   d^.d := 'd'; d^.x := 13; d^.y := 23;
26
27   { link nodes }
28   a^.next := b;
29   b^.next := c;
30   c^.next := d;
31   d^.next := nil;
32
33   p := a;  //p := @a;
34
35   writeln('p',#9,#9, #9, #9,#9, 'd',
36         #9, 'x', #9, 'y', #9, 'next');
37
38   while p <> nil do
39   begin
40     writeln(
41       HexStr( p), #9,
42       p^.d, #9,
43       p^.x, #9,
44       p^.y, #9,
45       HexStr( p^.next )
46     );
47     p := p^.next;
48   end;
49
50   dispose(a);
51   dispose(b);
52   dispose(c);
53   dispose(d);
54 end.
```

HOMEWORK1 : จากตัวอย่างที่ 1 ให้ นศแปลงโปรแกรมโดยใช้เทคนิค structure

สมมุติว่าข้อมูล อาจารย์คณะวิทยาศาสตร์มี n ดังนี้

เพศ (sex) : ชาย = 1 , หญิง = 2

อายุ (age) : ? ปี

สถานภาพสมรส (status) : โสด = 1 , แต่งงาน = 2 , อื่นๆ = 2

ปริญญาสูง (degree) : ตรี = 1 , โท = 2 , เอก = 3

จำนวนปีที่สอน (experience) : ? ปี

Number of teacher = 2
Sex = 1
Age = 35
Status 1
Degree =2
Experient = 10

Sex = 1
Age = 35
Status 1
Degree =2
Experient = 10

จงเขียน

1. **โปรแกรมรับข้อมูลจำนวนอาจารย์ n คน ของคณะวิทยาศาสตร์ และข้อมูลของอาจารย์**
2. **หาจำนวนปีเฉลี่ยของ อายุ และ จำนวนปีทีสอน**

AVEREAGE AGE = xx.xx Years

AVEREAGE Experience = XX.XX Years

Ex 1 จงใช้โครงสร้างข้อมูลแบบ record

น.ศ. n คน แต่ละคนสอบ 3 ครั้งในวิชาเขียนโปรแกรม P1,P2,P3 และได้คะแนนสอบ 3 ครั้งในวิชาคณิตศาสตร์ (M1,M2,M3)

จงเขียนโปรแกรมรับจำนวนนักศึกษา, ชื่อ ,คะแนน,ของแต่ละวิชา ในแต่ละครั้ง เพื่อหาคะแนนเฉลี่ยของ น.ศ. แต่ละคนแต่ละวิชา รวมทั้งคนที่มีคะแนนเฉลี่ยวิชาคณิตศาสตร์สูงสุด

น.ศ. คนที่ 1

20 30 20	คณิตศาสตร์
40 50 20	เขียนโปรแกรม

น.ศ. คนที่ 2

35 30 25	คณิตศาสตร์
32 20 50	เขียนโปรแกรม

Dynamic Memory Allocation Problems

- Dangling pointers (dangerous)
  - A pointer points to a heap-dynamic variable that has been deallocated (free memory)
    → has pointer but no storage
  - What happen when deferencing a dangling pointer?
- memory leakage (Lost heap-dynamic variable)
  - An allocated heap-dynamic variable that is no longer accessible to the user program (often called *garbage*)
    → has storage but no pointer
    - The process of losing heap-dynamic variables is called *memory leakage*

What does memory leaking mean?

- Definition:
  - Particular type of unused memory, unable to release
- Common:
  - Refer to any unwanted increase in memory usage
- Different from memory fragmentation

Leak1.pas

```
program Leak1;
type
  PInt = ^Integer;
var
  p: PInt;
begin
  New(p);
  p^ := 10;
  { ลืม Dispose(p); }
end.
```

Leak2.pas

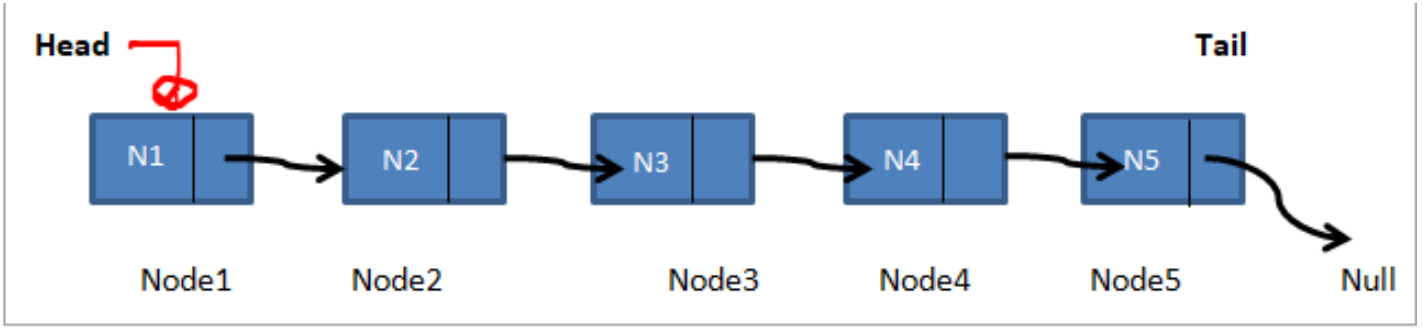
```
program Leak2;
type
  PInt = ^Integer;
var
  p: PInt;
begin
  New(p);
  p^ := 5;

  New(p);  { pointer เดิมหายไป }
  p^ := 10;

  Dispose(p);
end.
```

```
program MemoryLeakDemo;

type
  TString10 = STRING[10];
  PString10 = ^TString10;

function GenString(): PString10;
var
  p: PString10;
begin
  New(p);
  p^ := 'Hi';          { ตั้งค่า string ถูกต้อง }
  exit(p);     { ownership → caller }
end;

var
  s: PString10;
begin
  WriteLn( GenString()^ );
  s := GenString();
  WriteLn( s^ );
  Dispose(s);
end.
```

Leak3.pas

Advance Pointer

Linked List abstract data type (ADT)



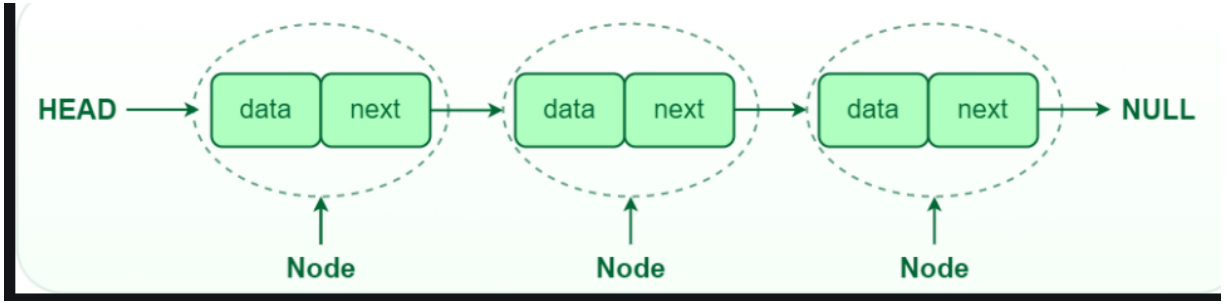
Suppose the data items on the list are ordered by ascending

Operations :

1. Add
2. Find
3. Remove
4. Show

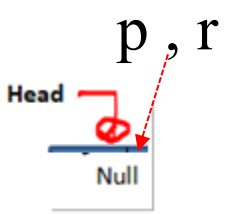
simLL.pas

```
type
  PNode = ^TNode;
  TNode = record
    d: Integer;
    next: PNode;
  end;
```

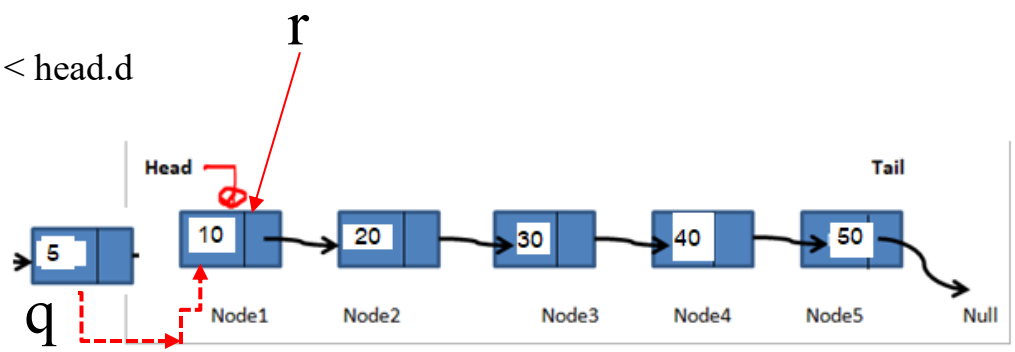


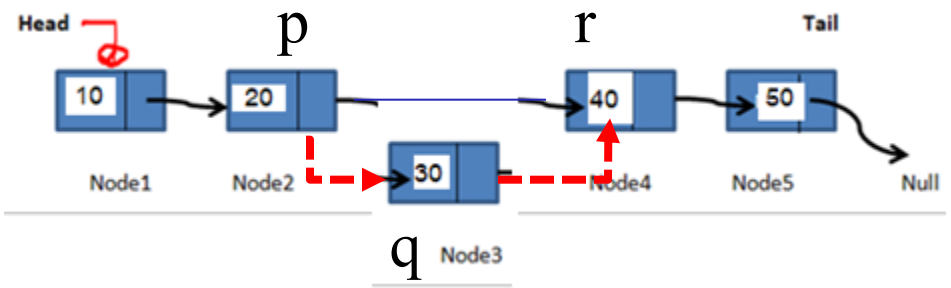
simLL.pas

```
9  procedure AddNode( var Head:PNode;  d:Integer);
10 var p,q,r:PNode;
11 begin
12   new(q);
13   q^.d := d;
14   q^.next := nil;
15   if( Head = Nil)  then
16   begin
17     Head := q;
18     exit();
19   end;
20
21   if(Head^.d > d ) then
22   begin
23     q^.next := head;
24     head := q;
25     exit();
26   end;
27
28   r := Head^.next;
29   p := Head;
30   while(r <> nil) do
31   begin
32     if( r^.d > d ) then
33       break;
34     p := r;
35     r := r^.next;
36   end;
37
38   p^.next := q;
39   q^.next :=r;
40 end;
```



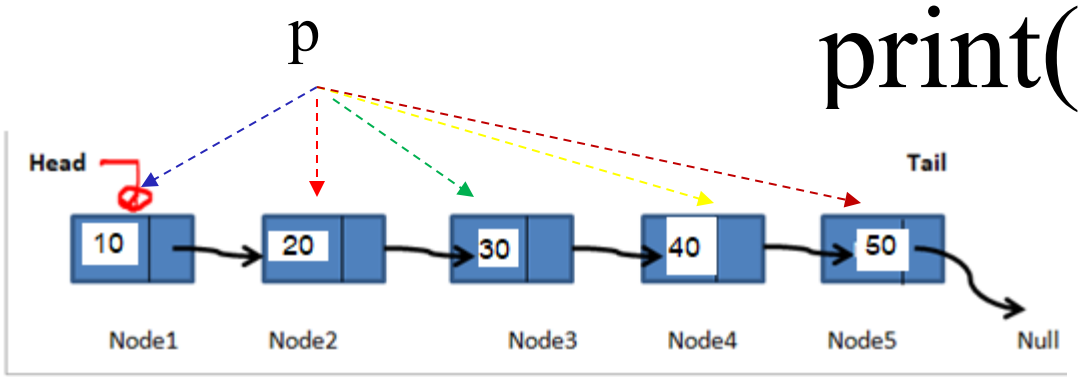
d < head.d



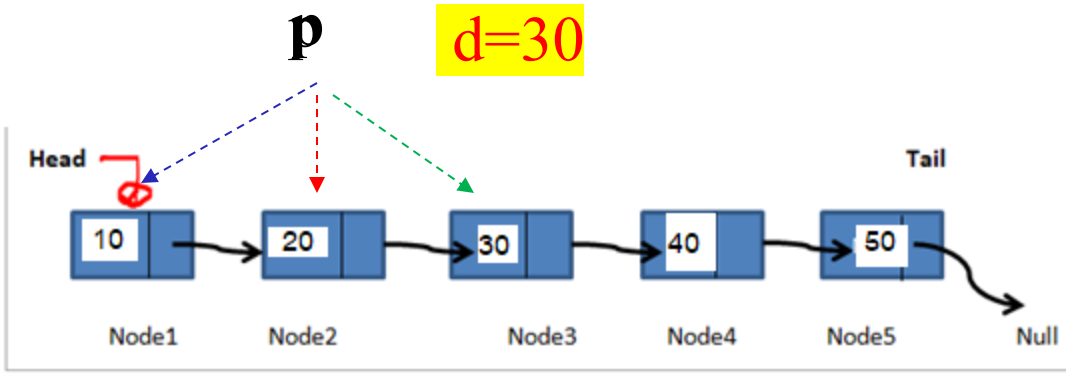



```
51 var
52   Head:PNode;
53 begin
54   Head := nil;
55   AddNode(Head,40);
56   AddNode(Head,20);
57   AddNode(Head,10);
58   AddNode(Head,30);
59   AddNode(Head,50);
60   Print(Head);
61 end.
```

simLL.pas




```
42  procedure Print( Head:PNode);
43  begin
44    while(Head <> Nil) do
45    begin
46      writeln( Head^.d);
47      Head := Head^.next;
48    end;
49  end;
```



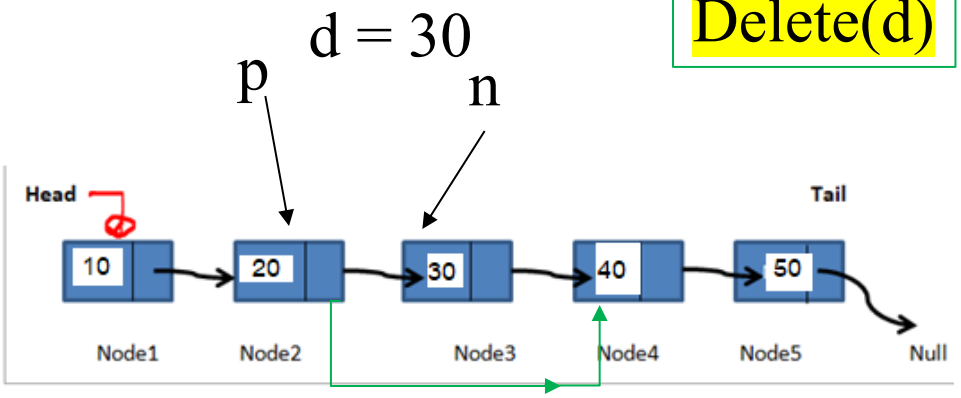

```
Writeln( Find( head,20) );
Writeln( Find( head,11) );
```

true
false

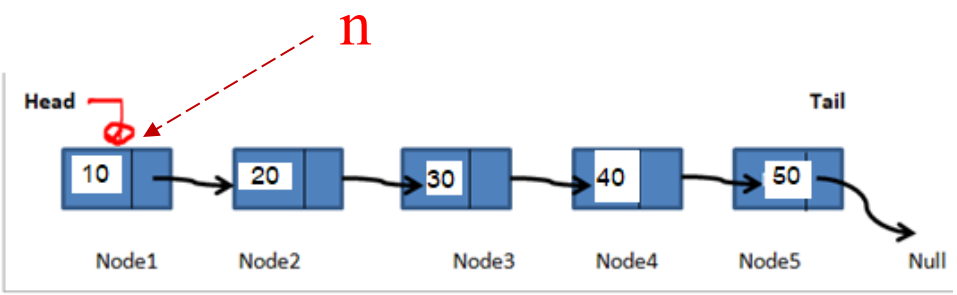
ให้ นศ.เขียนfunction
Find()

Delete(d)

ให้ นศ.เขียนfunction
Delete()

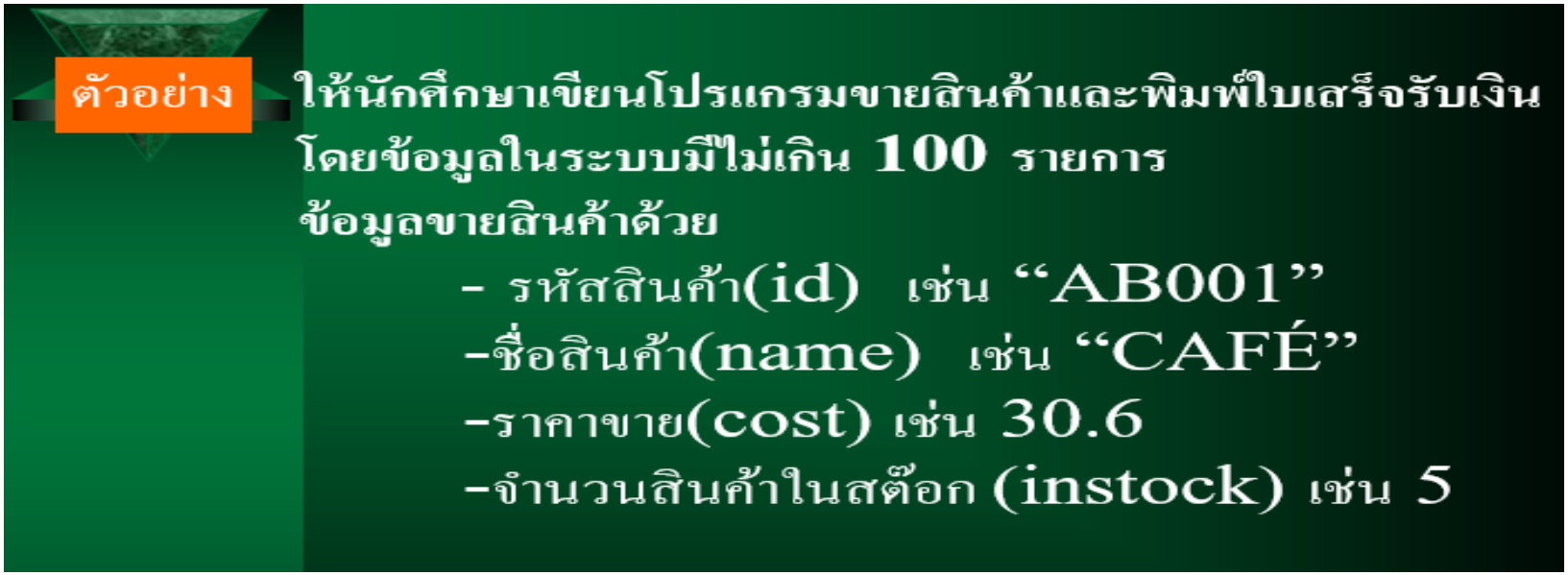


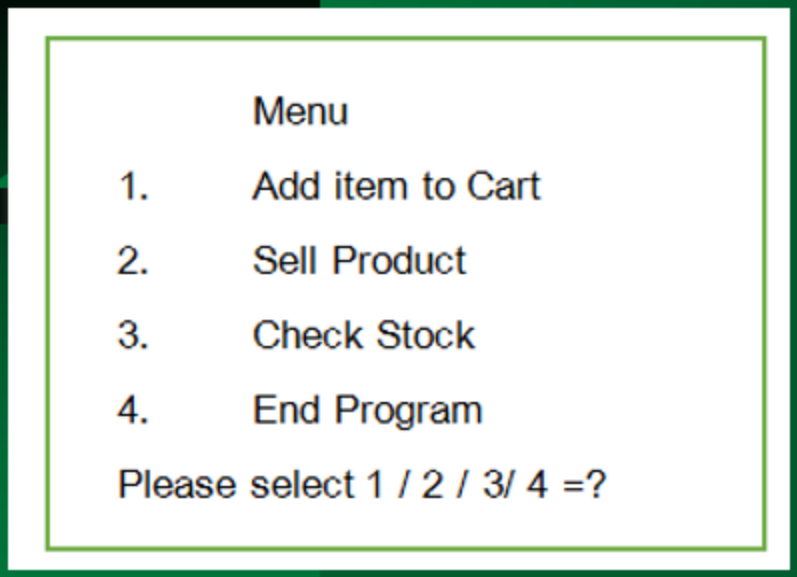
d = 10 p = NULL



Writeln(delete(head,20)); true
Writeln(delete(head,11)); false

Rewrite the code using Linked List





กด 1

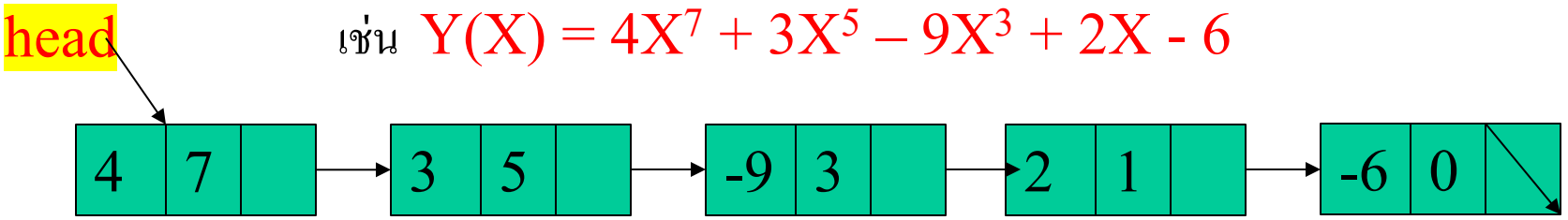
Input your product data:
id : AB001
name : CAFE
Cost : 30.6
Stock : 5

กด 2

Input your product id: AB001
Input your sell volume : 2
name : CAFE
Cost : 30.6
stock : 3
Pay : 61.2 bath

Ex1: จงเขียนโปรแกรมเก็บสมการโพลิโนเมียล และคำนวณค่า

เช่น $Y(X) = 4X^7 + 3X^5 - 9X^3 + 2X - 6$



โหนด	coef ค่าสัมประสิทธิ์	power ค่ายกกำลัง	
$+4X^7$	+4	7	
$+3X^5$	+3	5	
$-9X^3$	-9	3	
$+2X$	+2	1	
-6	-6	0	โดย $X^0 = 1$

ถ้า ค่ายกกำลังมีค่าเท่ากับ 0 ให้หยุดรับข้อมูล โดยการป้อนข้อมูลให้ผู้ใช้ป้อนข้อมูลเรียงตามลำดับค่าของเลขยกกำลังสูง

ตัวอย่างการรับข้อมูล

ถ้า ค่ายกกำลังมีค่าเท่ากับ 0 ให้หยุดรับข้อมูล

0. add coef
1.caculate X
2.exit

ปรากฏดังรูป

```
coef=?4
power=?7
coef=?-9
power=?3
coef=?3
power=?5
coef=?-6
power=?0
coef=?2
power=?1
coef=?0
+4X^7+3X^5-9X^3+2X^1-6
```

ป้อนค่าสัมประสิทธิ์และค่ายกกำลัง

ป้อนค่าสัมประสิทธิ์เป็น 0
หยุดการรับข้อมูล

ผลการทำงานจะเรียงลำดับโหนด
ของข้อมูลตามกำลังจากมากไปน้อย

ตัวอย่างการใช้ pointer ร่วมกับ subroutine

```
1  program PointerRecordFunctionDemo;
2
3  type
4    TPoint = record
5      x : Integer;
6      y : Integer;
7    end;
8    PTPoint = ^TPoint;
9    PInt    = ^Integer;
10
11 function GetInput : TPoint;
12 var
13   p1 : TPoint;
14 begin
15   p1.x := 20;
16   p1.y := 30;
17   exit( p1 );
18 end;
19
20 var
21   px : TPoint;
22 function GetInputPtr():PTPoint;
23 begin
24   px.x := 120;
25   px.y := 130;
26   exit( @px );
27 end;
```

```
29 procedure Output(p2 : PTPoint; x : PInt);
30 begin
31   writeln('1.p2^.x = ', p2^.x, ' , p2^.y = ', p2^.y);
32   p2^.x := p2^.x +100;
33   p2^.y := p2^.y +100;
34   writeln('2.p2^.x = ', p2^.x, ' , p2^.y = ', p2^.y);
35   writeln('3.x^ = ', x^);
36   x^ := x^ +100;
37   writeln('4.x^ = ', x^);
38
39 end;
40
41 var
42   p1 : TPoint;
43   p2 : PTPoint;
44   z  : Integer;
45
46 begin
47   z  := 10;
48   p1 := GetInput();
49   Output(@p1, @z);
50
51   writeln('5.p1.x = ', p1.x, ' , p1,y = ', p1.y);
52   writeln('6.z = ', z);
53
54   p2 := GetInputPtr();
55   writeln('7.p2^.x = ', p2^.x, ' , p2^.y = ', p2^.y);
56
57 end.
```

Output:

```
1.p2^.x = 20 , p2^.y = 30
2.p2^.x = 120 , p2^.y = 130
3.x^ = 10
4.x^ = 110
5.p1.x = 120 , p1,y = 130
6.z = 110
7.p2^.x = 120 , p2^.y = 130
```

BnkAcc.pas

แบบฝึกหัด 2

จงเขียนโปรแกรมต้องการนำระเบียนลูกค้า(customer) ของสหกรณ์ออมทรัพย์รามคำแหง ที่รองรับการจัดเก็บข้อมูลไม่เกิน 500 คน ระเบียนลูกค้าประกอบด้วย รหัสบัญชี(id) ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก (Deposit)

จงแปลงโปรแกรมจาก array ปกติให้เป็น dynamic allocation

Menu

1. Open a new Account
2. Deposit
3. Withdraw
4. Compound interest
5. Quit

Please select 1 / 2 / 3 / 4 / 5 =?

เลือก 1 เป็นการเปิดบัญชีใหม่

ระบบจะแสดงหน้าจอให้ผู้ใช้ ป้อนรหัสบัญชี(id) ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก (Deposit) โดยรหัสบัญชีต้องไม่ซ้ำกัน

เลือก 2 เป็นการฝากเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะตรวจสอบโดยการค้นหาระเบียนลูกค้า เพื่อให้ผู้ใช้งานป้อนยอดฝากเพื่อปรับปรุงบัญชีให้ถูกต้อง

เลือก 3 เป็นการถอนเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะตรวจสอบเพื่อ

ค้นหาระเบียนลูกค้า โดยการถอนเงินลูกต้องมีเงินติด

บัญชีอย่างน้อย 100 บาท ระบบทำการปรับปรุงบัญชีให้ถูกต้อง

เลือก 4 การคิดดอกเบี้ยทบต้น

1. โดยระบบจะให้ผู้ใช้ป้อนอัตราดอกเบี้ยต่อปี(rate) ทาง

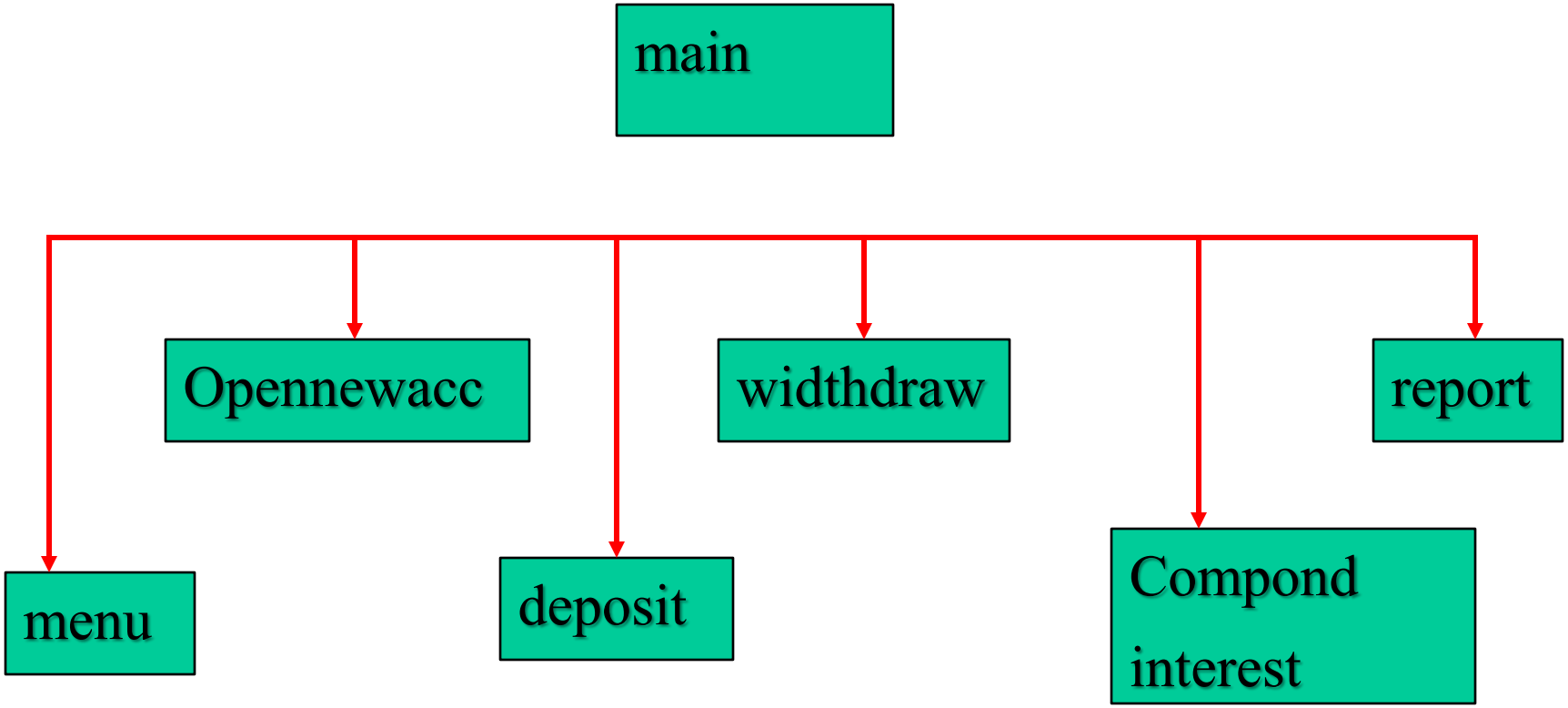
แป้นพิมพ์ เพื่อคิดดอกเบี้ยทบต้นของทุกบัญชี

2. โปรแกรมจะแสดงเงินฝากเก่า , ดอกเบี้ยที่ได้รับ และเงินฝากใหม่

เงินฝากใหม่ = เงินฝากเดิม+ดอกเบี้ยที่ได้

ดอกเบี้ยที่ได้ =อัตราดอกเบี้ยที่ได้*เงินฝากเดิม

เลือก 5 เป็นการจบการทำงานซ้ำ


main
Opennewacc
widthdraw
report
menu
deposit
Compond interest

Apirak :

main.pas

```
1  program BankAccount;
2
3  type
4    Account = record
5      id: string[10];
6      name: string[100];
7      addr: string[100];
8      deposit: real;
9    end;
10
11   AccountDataType = array[1..500] of Account;
12
13 function getChoice:integer;
14 var
15   choice:integer;
16 begin
17     writeln;
18     writeln('1. Open Acc');
19     writeln('2. Deposit Acc');
20     writeln('3. Withdraw Acc');
21     writeln('4. Compute Interest');
22     writeln('5. Exit');
23     write('Enter choice: ');
24     readln(choice);
25     exit(choice);
26 end;
```

BackAccV2.pas

แก้ไขให้เป็น dynamic
โดยใช้ new & dispose

BackAccV2.pas

```
28 Procedure OpenAcc( var cust:AccountDataType ; var n:integer );
29 var
30       id: string[10];
31       bfound: boolean;
32       i:integer;
33 begin
34     writeln;
35     writeln('Do Open Acc');
36     write('id Acc =? ');
37     readln(id);
38     bfound := False;
39
40     for i := 1 to n do
41     begin
42     if cust[i].id = id then
43     begin
44       bfound := True;
45       break;
46     end; // end if
47     end; //end for
48
49     if  bfound = False then
50     begin
51         n := n + 1;
52         cust[n].id := id;
53         write('name Acc =? ');
54         readln(cust[n].name);
55         write('addr Acc =? ');
56         readln(cust[n].addr);
57         write('deposit Acc =? ');
58         readln(cust[n].deposit);
59     end
60     else
61         writeln('id Acc found = ', id);
62 end;
```

BackAccV2.pas

```
64  Procedure Deposit( var cust:AccountDataType ; var n:integer );
65  var
66        id: string[10];
67        bfound: boolean;
68        deposit: real;
69        i:integer;
70  begin
71    writeln;
72    writeln('Do Deposit Acc');
73    write('id Acc =? ');
74    readln(id);
75
76    bfound := False;
77    for i := 1 to n do
78    begin
79      if cust[i].id = id then
80      begin
81        bfound := True;
82        break;
83      end; //end if
84    end; // end for
85
86    if bfound = False then
87      writeln('id Acc not found = ', id)
88    else
89     begin
90        writeln('Before deposit Acc = ', cust[i].deposit:0:2);
91        write('New deposit Acc =? ');
92        readln(deposit);
93        cust[i].deposit := cust[i].deposit + deposit;
94        writeln('After deposit Acc = ', cust[i].deposit:0:2);
95     end;
96  end; //case 2
```

```
98
99  var
100   cust: AccountDataType;
101   n, choice, i: integer;
102
103
104 begin
105   n := 0;
106   repeat
107     choice :=  getChoice();
108     case choice of
109       1: OpenAcc( cust , n );
110       2: Deposit( cust ,n );
111       3:
112         begin
113           writeln('Do Withdraw Acc');
114         end;
115       4:
116         begin
117           writeln('Do Interest Acc');
118         end;
119     end;
120
121   until choice = 5;
122 end.
123
```

```
1. Open Acc
2. Deposit Acc
3. Withdraw Acc
4. Compute Interest
5. Exit
Enter choice: 2

Do Deposit Acc
id Acc =? i003
id Acc not found = i003

1. Open Acc
2. Deposit Acc
3. Withdraw Acc
4. Compute Interest
5. Exit
Enter choice: 2

Do Deposit Acc
id Acc =? i001
Before deposit Acc = 330.00
New deposit Acc =? 20
After deposit Acc = 350.00

1. Open Acc
2. Deposit Acc
3. Withdraw Acc
4. Compute Interest
5. Exit
Enter choice: 5
```

Apirak :